

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



Le Minh Tam

**Real-Time Rigging and Secondary Motion
for Sketch-Based 3D Characters**

BACHELOR'S THESIS

Major: Information Systems

HANOI - 2026

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



Le Minh Tam

**Real-Time Rigging and Secondary Motion
for Sketch-Based 3D Characters**

BACHELOR'S THESIS
Major: Information Systems

Supervisor: Dr. Ma Thi Chau

HANOI - 2026

Acknowledgements

I would like to start by expressing my deepest gratitude to Dr. Ma Thi Chau. Her mentorship has been the steady thread throughout my academic journey, from my first steps in the Student Scientific Research Conference to the ICCSA 2025 conference and finally the completion of this thesis. I am incredibly grateful for her patience and the expert insights that helped me navigate every technical hurdle.

I also want to thank the faculty at the University of Engineering and Technology. The knowledge and support I received over these past four years provided the essential foundation for this research.

Finally, a special thank you to my family and friends for their unwavering encouragement. In particular, I am deeply indebted to Nhi, Hân, and Bảo, who are exceptionally talented artists, as well as my close friends Hương and Hiếu and my younger brother. By serving as the “guinea pigs” for my informal experiments, their artistic intuition and honest feedback were vital in refining the final product and bringing a human perspective to the work.

While I have put my heart into this research, I recognize that there is always room for growth. I look forward to the committee’s guidance and suggestions to help me further improve this study.

Declaration of Authorship

My name is **Le Minh Tam**, a student of class **QH-2022-I/CQ-IS**, Faculty of Information Technology, VNU University of Engineering and Technology.

I hereby declare that this thesis is my own research work, conducted under the supervision of Dr. Ma Thi Chau. The findings and data presented herein are truthful, original, and have not been previously published or plagiarized from any other sources. All citations, references, and borrowed information have been clearly identified and documented to ensure academic objectivity and integrity.

I accept full responsibility before the University's Disciplinary Board and current laws for any instance of fraud or academic dishonesty.

Hanoi, May 21, 2026

Author

Le Minh Tam

Statement on AI Assistance

In the process of completing this thesis, I utilized artificial intelligence assistants, specifically Claude 4.5 Sonnet and Gemini 3.0 Pro.

The assistance included improving English grammar, sentence clarity, academic expression, document organization, LaTeX formatting, explaining coding errors, supporting debugging, and suggesting possible improvements to code structure and implementation.

The use of AI-based tools was strictly limited to support and refinement. The core research ideas, experimental design, data processing, model implementation, result analysis, and final conclusions were independently developed, performed, checked, and verified by myself. I am fully responsible for all content, source code, experiments, results, and interpretations presented in this thesis.

Hanoi, May 21, 2026

Author

Le Minh Tam

Abstract

Traditional 3D character creation is often too technical, forcing artists to go through separate stages like modeling, rigging, and animation. While current sketch-based approaches give users more creative freedom, they usually focus on static modeling and lack a unified environment that integrates both rigging and physics-based animation. Building upon this existing research, this thesis proposes a user-friendly framework connecting sketching directly with physics-based movement. This thesis delivers three main research contributions: (i) a workflow for sketching a full character with real-time rigging, automatically creating a skeleton so the model can move right away; (ii) a mass-spring system integration that enables automatic secondary motion, allowing characters to mimic different materials without manual keyframing; and (iii) additional interactive features for real-time editing and animation. This framework provides immediate visual feedback and allows easy adjustments to physics settings. It helps beginners approach 3D animation easily, while offering professionals a playground for ideation and prototyping. In short, it lets all creators focus on their ideas instead of technical setup, maintaining their creative flow.

Keywords: *Sketch-based Modeling, Real-time Skeletonization, Mass-spring System, Real-time Secondary Animation.*

Contents

Acknowledgements	
Declaration of Authorship	i
Statement on AI Assistance	ii
Abstract	iii
Contents	iv
List of Figures	vi
List of Tables	vii
List of Abbreviations	viii
Chapter 1 Introduction	1
1.1 Problem Statement	1
1.2 Research Questions	2
1.3 Research Contributions	3
1.4 Thesis Structure	3
Chapter 2 Theoretical Background	5
2.1 Related Work	5
2.1.1 Sketch-based 3D Modeling	5
2.1.2 Mass-spring System	6
2.2 Geometric Methods for Sketch-based 3D Modeling	7
2.3 Fast Simulation of Mass-spring Systems	9
2.3.1 Fundamental Theory	10
2.3.2 Reformulating the Energy Potential	12
2.4 Harmonic Weight Calculation	15

2.5 Linear Blend Skinning Algorithm	17
Chapter 3 Proposed Methodology	19
3.1 System Overview	19
3.2 Mesh Stitching and Real-time Rigging	21
3.3 Integrating Mass-Spring Systems for Dynamic Secondary Motion	24
3.3.1 Building Matrices	25
3.3.2 Local Update	26
3.3.3 Global Update	26
3.3.4 Convergence Check	27
3.4 Improving User Interaction	28
3.4.1 Painting on Model Surfaces	28
3.4.2 Manual Skeleton Adjustment	29
3.4.3 Regional Control Mechanism	29
3.4.4 Animation Mode	30
Chapter 4 Results and Discussion	31
4.1 System Implementation and Experimental Setup	31
4.2 Visual Results of Generated Models	32
4.3 Computational Performance Analysis	36
4.4 Rigging and Dynamic Motion Analysis	37
4.4.1 Motion Comparison	37
4.4.2 Effects of Parameters	38
4.5 User Feedback	40
4.6 Limitations and Future Work	43
Conclusion	45
List of Author's Publications	46
Bibliography	47
Chapter A User Experience Survey Using Likert Scale	51

List of Figures

2.1	Teddy’s triangulation and chordal axis transformation process.	8
2.2	Pruning algorithm for secondary branches based on the semicircle criterion.	9
2.3	Spine elevation and elliptical cross-section creation.	9
2.4	Three states of a mass-spring system: equilibrium, stretched, and compressed.	13
2.5	Geometric interpretation of the optimization process.	14
2.6	Visual representation of weights on the character’s surface mesh.	16
2.7	The Linear Blend Skinning process for mesh deformation.	18
3.1	System overview workflow for constructing 3D models from sketches.	20
3.2	Hierarchical tree structure for managing the construction sequence and mesh stitching logic.	21
3.3	Motion generation via trajectory sketching	30
4.1	The graphical user interface of the developed framework.	31
4.2	Some models created by users from the system	33
4.3	Comparison of system performance with and without a mass-spring system on the same model.	38
4.4	Comparison of the influence of spring stiffness k and damping coefficient k_d on model motion effects.	39
4.5	Marked point on the rabbit ear (left) and a plot showing the displacement of that point when adjusting different physical parameters (right).	39
4.6	Survey results evaluating user experience across three core criteria.	41

List of Tables

4.1	Comparison of model results from the original source code, student-developed models, Teddy, and MonsterMash.	34
4.2	Feature comparison between the proposed system and related research . .	35
4.3	Model Complexity and Performance Statistics Across Phases	36
4.4	Effects of Stiffness (k) and Damping (k_d) on Character Motion	40

List of Abbreviations

Abbr.	Full Term	Description
LBS	Linear Blend Skinning	A standard technique to simulate surface mesh deformation based on skeletal movement.
CDT	Constrained Delaunay Triangulation	A triangulation method used to classify mesh triangles into terminal, sleeve, or junction types.
BFS	Breadth-First Search	An algorithm used to process and expand adjacent mesh layers according to the tree structure.
UV	UV Mapping	The process of projecting 3D mesh vertex coordinates onto 2D image space for texturing.
FPS	Frames Per Second	The simulation rate used for the mass-spring system, set at 60 FPS.
PNG	Portable Network Graphics	An image format with transparency used for automatic contour extraction and mapping.
GPU	Graphics Processing Unit	Hardware utilized for real-time texture map updates to optimize memory and performance.

Chapter 1

Introduction

1.1 Problem Statement

In its early days, 3D technology carried the promise of a more efficient creative process by shifting heavy manual tasks onto computers. The hope was that instead of drawing thousands of frames by hand, artists could focus on their vision while the software handled everything from lighting to physics. By removing the need for physical skill, this shift was meant to let anyone bring complex ideas to life within a 3D space.

In reality, however, current workflows have introduced significant technical hurdles that often hinder the creative process. Beyond just having an aesthetic sense, modern 3D artists now find they need a fundamental knowledge of computer architecture and mathematical parameters. To transform a single idea into a moving character, an artist must navigate a long series of complex intermediate steps, including modeling, rigging, and adjusting surface parameters before ever reaching the animation stage. Because the workflow is fragmented into so many of these specialized phases, there is almost no room to freestyle or experiment. Instead of acting immediately on a creative whim, artists frequently have to stop to handle technical chores or fix software bugs. Suffering from these constant interruptions, the original creative spark often cools down and fades away before the work is even finished.

The restriction becomes even clearer when we compare animation to other art forms. Within poetry, music, or painting, there exists a level of spontaneity rarely found in 3D work, largely because those creators can reach a peak state only when the barriers between an idea and its execution disappear. Without this direct connection between a concept and its final result, creativity loses its effectiveness. According to Csikszentmihalyi, people reach their most creative state only when they enter a flow that is free from technical obstacles and, more importantly, the fear of making a mistake [1].

Even with the arrival of groundbreaking tools like Teddy [2], MonsterMash [3], or Rodin AI², the standard workflow remains mostly strict and unchanged. Evidence of this de-

²<https://hyper3d.ai/>

mand appears in user surveys conducted by the MonsterMash research team. By studying a group of experienced professionals, of whom 80% work in 2D or 3D animation and 60% in traditional hand-drawn animation, researchers found overwhelmingly positive feedback regarding the efficiency of sketch-based methods [3]. Regardless of their skill level, these artists share a common desire to transform hand-drawn strokes into 3D characters as quickly as possible to begin the animation process. Seeing high-quality results immediately does more than just save effort, as it provides the necessary inspiration to spark new ideas during the creative process. Because flexibility and speed are the main goals during early sketching, having access to these tools becomes vital. Rather than getting stuck behind technical barriers at the very start, creators can stay focused on the work itself by turning abstract thoughts into motion almost instantly. Additionally, many artists prefer letting the computer handle physical laws instead of spending hours fixing every tiny detail by hand. By using gravity and momentum, secondary motions can automatically simulate soft, squishy materials and dynamic jiggle physics. Cutting out these technical chores keeps the creator focused on the art itself.

1.2 Research Questions

Considering the technical hurdles previously described, this study seeks to answer how a more intuitive 3D workflow can be built to keep the creative spark alive. The goal is to provide an environment where beginners can model and animate without feeling overwhelmed, while professionals can use the same tools as a high-speed playground for testing new concepts.

- **RQ1:** How can a sketch-based system be designed to combine modeling and rigging into a single, real-time step? Rather than treating these as separate technical phases, the goal is to turn a 2D drawing into a ready-to-move 3D character instantly.
- **RQ2:** In what ways can automated secondary motion be integrated for dynamic movement without manual effort? How can we use gravity and momentum to simulate different materials, from soft, squishy shapes to hard, solid ones?
- **RQ3:** Which specific features are necessary to improve interactivity so that the artist stays in a state of flow?

1.3 Research Contributions

This research uses the algorithm introduced by Igarashi et al. in the Teddy paper [2] as a foundation. With this method, a user simply sketches a shape, and the software automatically turns that stroke into a 3D model. While the original workflow is effective for building characters piece-by-piece, this study offers a different approach designed for users who prefer to sketch an entire character before generating the 3D model.

By expanding on the existing system, this research provides three main contributions:

1. The proposed system introduces a workflow that allows for sketching a full character in one go with real-time rigging included. With this approach, users are able to focus on the character's entire silhouette and structure at once. The system then automatically creates a skeleton that matches the model, ensuring the character is ready to move immediately after it is generated.
2. The integration of a mass-spring system into the skeleton enables the automatic computation of secondary motion. Characters can now mimic various materials, ranging from rigid objects to soft, squishy shapes. By adjusting physical parameters, the system is able to produce dynamic animation effects, such as follow-through, slow-in, and slow-out [4], without the need for manual keyframing.
3. Additional interactive features enhance the user experience by allowing for model editing and animation in real time. These capabilities include drawing directly on the character for decoration, refining the skeleton structure, and animating the figure through intuitive guiding. While the proposed algorithms deliver the foundational value of this research, their practical significance would be lost without an intuitive and responsive interaction framework. By granting artists creative freedom and providing immediate, visually acceptable models, the system effectively bridges the gap between complex mathematics and artistic intent, ultimately eliminating the technical barriers originally addressed.

1.4 Thesis Structure

The thesis is structured into four primary chapters and a concluding section:

- **Chapter 1: Introduction** introduces the research, outlining the problem statement regarding the technical challenges in 3D content creation, and defining three core research questions alongside the main contributions.
- **Chapter 2: Theoretical Background** establishes the theoretical foundation, reviewing related literature in sketch-based modeling and the fundamental physics governing fast mass-spring system simulation and skinning algorithms.
- **Chapter 3: Proposed Methodology** details the proposed framework, describing the technical implementation of real-time mesh stitching, automated skeletonization, and the integration of dynamic secondary motion.
- **Chapter 4: Results and Discussion** presents the experimental results and evaluation, encompassing a system performance analysis, a comparative discussion with existing frameworks such as Teddy and MonsterMash, and a synthesis of user feedback gathered from both professional and non-professional cohorts.
- **Conclusion** finalizes the thesis by summarizing the research achievements, addressing current limitations concerning spatial depth, and outlining avenues for future work.

Chapter 2

Theoretical Background

2.1 Related Work

2.1.1 Sketch-based 3D Modeling

As a pioneering system, Teddy [2] streamlines 3D design by allowing users to generate freeform shapes with just a few strokes. Instead of using complex tools, creators draw 2D outlines that the system automatically inflates and merges into a single mesh. Although this method reduces the technical hurdles of traditional modeling, artists must still construct characters piece by piece.

Over time, numerous follow-up works [5–10] have expanded on this approach, enabling the creation of more complex shapes and making sketch-based modeling more comprehensive. Despite these improvements, the tools remain quite technical. They often require complex operations that slow down the creative flow, making them feel like tools for engineers instead of artists.

For purely spontaneous creativity, staying within a 2D mindset and interaction remains the most appealing choice. The SmoothSketch system [11] addresses this by automatically analyzing strokes to infer shapes and how parts should connect. Although the system is limited to building simple shapes, it shows that modeling can be fully automated using only 2D operations. Building on this idea, MonsterMash [3] is a project sponsored by Google Research that introduces a single-view approach to casual 3D modeling and animation. It combines 3D inflation with a rigidity-preserving deformation algorithm called ARAP-L to produce a smooth 3D mesh ready for animation immediately. Moreover, the model can be animated from a single viewpoint without causing models to overlap or collide, which was a common issue in previous methods.

Currently, with the rapid growth of deep learning, data-driven approaches are opening up new possibilities [12–14]. However, deep learning methods often require several minutes to process, which prevents a true real-time interaction experience. Furthermore, if the output is not what the user expected, it is difficult to intervene or precisely control

the inference process to make adjustments.

Both geometric and data-driven approaches share a common limitation. It is that their output often remains a static 3D mesh. Making the model move requires separate rigging afterward. This rig consists of a skeleton, which is a cycle-free graph where nodes represent joints and edges represent bones [15]. Because of the complexity of the manual process, automatic skeletonization and rigging techniques have also been extensively researched. Notable examples include Pinocchio [16], which adapts a skeleton to the character’s surface, allowing motion data to animate the model. RigMesh [15] extends this concept by creating a rig for each sketched part in real-time and updating it as parts are merged or cut. More recent approaches have leveraged deep learning for further automation. RigNet [17] introduces a neural network that predicts a skeleton matching an animator’s expectations in joint placement and topology. For single-image rigging, Object Wake-up [18] reconstructs 3D objects and generates skeletons from 2D images using deep implicit functions.

Despite shortening the overall workflow, automatic skeletonization and rigging methods impose a heavy practical burden on creators. Most approaches are computationally slow, requiring minutes or hours to handle complex meshes. They also work like black boxes with no middle control points. If a user wants to fix just a single joint, they must restart the whole process from the beginning. Furthermore, these methods disrupt the creative flow. To avoid problems like parts overlapping or colliding, users have to constantly leave their simple 2D workspace to check the model from many different 3D angles.

2.1.2 Mass-spring System

Mass-spring systems have long been used in both scientific computing and physics-based animation to simulate movement and deformation [19]. These systems represent objects as networks of interconnected masses and springs, enabling realistic simulations of elasticity, bending, and other physical behaviors. Early models for cloth simulation [20, 21] demonstrated how simple physics-based structures could produce visually convincing motion. They have also been applied to approximate the deformation of volumetric solids [22] and linear elements such as hair [23, 24]. Spring bone dynamics, as seen in tools like Cartoon Animator 5¹ and VRM², provide simplified secondary motion effects, making

¹<https://www.reallusion.com/cartoon-animator/>

²<https://vrm.dev/en/>

them valuable for game development and AR/VR applications. Researchers have also improved these methods for greater stability and efficiency, with the Fast Mass-Spring method [25] introducing a more efficient solver for real-time deformable object simulation.

Hair and cloth simulation are two fields where mass-spring systems play a key role, especially at major studios like Pixar. According to an interview with The Atlantic¹, Merida’s curly hair in the movie *Brave* created a difficult physics problem. Traditional geometric models simply could not replicate the lively bounce or layered structure of the curls. To fix this, Pixar built a modified mass-spring model with core springs inside each strand. Physical laws programmed into the software govern the hair’s behavior. When Merida moves, gravity and inertia pull the curls down, but they immediately snap back like a spring. These internal springs act as a flexible frame, letting the hair react naturally while keeping its original shape. Working this way completely removes unnatural stretching and distortion.

In cloth simulation, mass-spring systems remain the foundational standard for objects like flags, capes, or character costumes [26]. The cloth structure is discretized into a grid of mass points, tightly linked by functional springs that precisely control three basic properties: stretching, shearing, and bending. These constraints ensure the fabric surface maintains consistency and help prevent model penetration in 3D space. Modern mass-spring systems often integrate with deep learning models to improve collision handling and generate more natural wrinkles [26–28].

2.2 Geometric Methods for Sketch-based 3D Modeling

This section summarizes the construction of a 3D polygonal mesh from sketched strokes using the Teddy system [2], which serves as the core method for this thesis. Technically, the system outputs a closed polygonal mesh that consistently remains topologically equivalent to a sphere.

The Teddy system constructs 3D polygon meshes from 2D sketches by first ensuring the input is topologically consistent with a sphere and mathematically refined. Every hand-drawn stroke is resampled into a smooth, uniform-edge polyline to eliminate noise before forming a closed planar polygon. The system then applies Constrained Delaunay

¹<https://www.theatlantic.com>

Triangulation, which classifies the resulting triangles into terminal, sleeve, or junction types based on their number of external edges. To establish a clean internal structure, the system identifies a chordal axis or spine by connecting the midpoints of internal edges, but this requires a rigorous pruning process to remove insignificant branches (see Figure 2.2). The complete triangulation and chordal axis transformation is depicted in Figure 2.1.

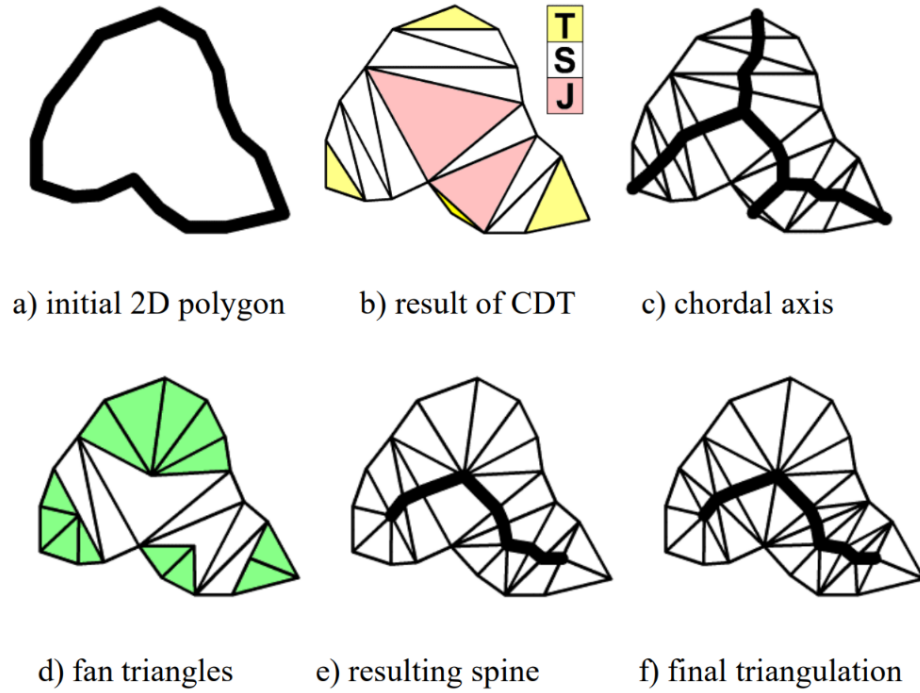


Figure 2.1: Teddy's triangulation and chordal axis transformation process.

The system prunes insignificant skeletal branches to eliminate surface artifacts such as bumps or spikes, thereby ensuring a smooth and sphere-equivalent 3D mesh. As illustrated in Figure 2.2, this pruning is governed by a semicircle criterion. Terminal triangles are merged with their neighbors if they fall within a semicircle drawn on their internal edge. Once the branches are trimmed, the area is re-triangulated into a fan-like mesh radiating from the spine.

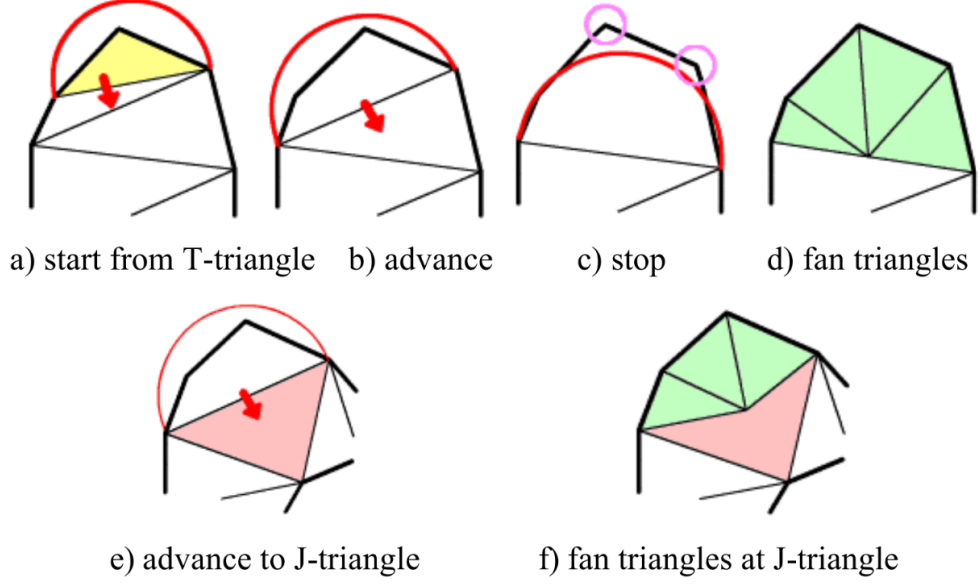


Figure 2.2: Pruning algorithm for secondary branches based on the semicircle criterion.

The transition to 3D space is achieved through an inflation-based elevation process (see Figure 2.3), where vertices on the spine are raised proportionally to their distance from the polygon's boundary. By warping internal edges into quarter-oval arcs and mirroring the resulting surface, the system generates a closed, symmetrical 3D volume. Finally, post-processing smoothing is executed to eliminate mesh defects, creating in a well-distributed and regularized 3D polygonal mesh.

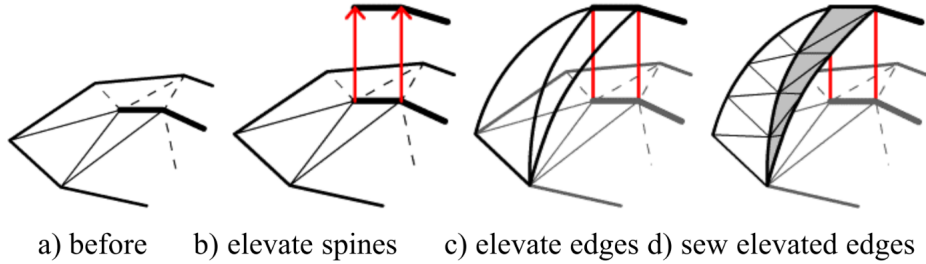


Figure 2.3: Spine elevation and elliptical cross-section creation.

2.3 Fast Simulation of Mass-spring Systems

This section reviews the key contributions by Liu et al. [25], which introduces a fast and stable method for time integration of mass-spring systems using a block coordinate descent solver. Their approach reformulates the implicit Euler method as an energy

minimization problem, introducing spring directions as auxiliary unknowns to decouple nonlinear terms and allow efficient pre-factorization of the system matrix.

2.3.1 Fundamental Theory

Before delving into the details of their proposed method, the thesis will first clarify some fundamental theory underlying mass-spring systems and implicit time integration.

Consider the model as a graph consisting of m points in 3D space:

- Each vertex p_i represents a point mass.
- Each edge represents a spring.

In a mass-spring simulation, each point starts with an initial position and initial velocity, which determine how it moves over time. According to Newtonian physics, if the current state of an object is known, its future state can be calculated by applying physical equations. This principle also holds in computer simulations, where the system updates positions and velocities at discrete time steps to approximate real-world motion. The key question to be addressed here is: **How can we determine the next positions of the vertices after a time step h ?**

The system evolves through a discrete sequence of time steps $t_1, t_2, \dots$ with a constant time step h . At each time step t_n , the system's configuration is represented as $\mathbf{p}_n \in \mathbb{R}^{3m}$, $\mathbf{p}_n$ is a vector containing the positions of all m points in 3D space at time t_n .

Assume that at time t , the current state $\mathbf{p}_t$ is known. For $t = 0$, these serve as the initial conditions for the entire simulation. For $t \geq 0$, they can still be considered the initial conditions for the subsequent time intervals. Denote $\mathbf{f}(\mathbf{p}_t)$ as the vector of forces acting on all particles $\mathbf{p}_i^t$ at time t , and $\mathbf{M} \in \mathbb{R}^{3m \times 3m}$ as the diagonal mass matrix.

Newton's Second Law states that the force $\mathbf{f} \in \mathbb{R}^3$ acting on an object is equal to the product of its mass and acceleration $\mathbf{a} \in \mathbb{R}^3$, as expressed in Equation (2.1):

$$\mathbf{M}\mathbf{a} = \mathbf{f} \tag{2.1}$$

We construct the animation by applying this equation to every vertex in the graph. The force $\mathbf{f}_i$ acting on vertex i is the sum of all forces from adjacent springs ij .

In the real world, an object's trajectory follows a continuous curve over time. However, in this simulation, we are only concerned with the discrete positions of the vertices at

specific time steps. We will construct discrete approximations using implicit Euler integration, tracking the system states through the node positions $\mathbf{p}$ and velocities $\mathbf{v}$. Given a time step size h , the state update rules are governed by the following system of equations:

$$\mathbf{p}_{n+1} = \mathbf{p}_n + h\mathbf{v}_{n+1} \quad (2.2)$$

$$\mathbf{v}_{n+1} = \mathbf{v}_n + h\mathbf{M}^{-1}\mathbf{f}(\mathbf{p}_{n+1}) \quad (2.3)$$

where $\mathbf{p}_n$ and $\mathbf{v}_n$ denote the position and velocity vectors at the current frame n , while $\mathbf{p}_{n+1}$ and $\mathbf{v}_{n+1}$ represent the target states for the subsequent time step. The term $\mathbf{M}^{-1}$ signifies the inverse mass matrix, and $\mathbf{f}(\mathbf{p}_{n+1})$ represents the net forces acting on the vertices.

Because the force vector in Equation (2.3) depends evaluated at the future position $\mathbf{p}_{n+1}$ rather than the current state $\mathbf{p}_n$, substituting Equation (2.3) into Equation (2.2) yields a non-linear system that must be solved implicitly at every time step.

Using the position update equation from implicit Euler integration, we can express velocity in terms of discrete differences.

Backward Difference Approximation The velocity at time step n is given by:

$$h\mathbf{v}_n = \mathbf{p}_n - \mathbf{p}_{n-1} \quad (2.4)$$

Forward Difference Approximation Similarly, the velocity at the next time step $n+1$ can be written as:

$$h\mathbf{v}_{n+1} = \mathbf{p}_{n+1} - \mathbf{p}_n \quad (2.5)$$

To eliminate velocity from the implicit Euler update, we multiply Equation (2.3) by h and substitute Equation (2.4) and Equation (2.5), leading to:

$$\mathbf{p}_{n+1} - 2\mathbf{p}_n + \mathbf{p}_{n-1} = h^2\mathbf{M}^{-1}\mathbf{f}(\mathbf{p}_{n+1}) \quad (2.6)$$

Since $\mathbf{f}(\mathbf{x})$ depends on the unknown future position $\mathbf{x} := \mathbf{p}_{n+1}$, this formulation presents a coupled non-linear problem. By substituting the velocity update rule into the position update rule, the non-linear system can be rewritten as an optimization problem. To simplify the notation for the optimization step, we define the target future position as $\mathbf{x} := \mathbf{p}_{n+1}$ and the inertial predictive position as $\mathbf{y} := 2\mathbf{p}_n - \mathbf{p}_{n-1}$. Multiplying both sides of the combined implicit equation by the mass matrix $\mathbf{M}$, we obtain a compact force-balance form:

$$\mathbf{M}(\mathbf{x} - \mathbf{y}) = h^2 \mathbf{f}(\mathbf{x}) \quad (2.7)$$

where $\mathbf{y}$ represents the explicit, unconstrained step predicted solely by inertia. Assuming the net forces are conservative and can be derived from a total potential energy function $E(\mathbf{x})$ such that $\mathbf{f}(\mathbf{x}) = -\nabla E(\mathbf{x})$, Equation (2.7) matches the exact first-order optimality condition ($\nabla g(\mathbf{x}) = 0$) of the global minimization objective expressed in Equation (2.8):

$$\min_{\mathbf{x}} g(\mathbf{x}) = \frac{1}{2}(\mathbf{x} - \mathbf{y})^T \mathbf{M}(\mathbf{x} - \mathbf{y}) + h^2 E(\mathbf{x}) \quad (2.8)$$

2.3.2 Reformulating the Energy Potential

As stated in the paper by Liu et al. [25], the main idea of their technique is to reformulate the energy potential E in a way that will allow us to employ a block coordinate descent method. In a mass-spring system, the elastic potential energy represents the energy stored in a deformed spring. According to Hooke's Law, the potential energy function is given by:

$$E(\mathbf{p}_i, \mathbf{p}_j) = \frac{1}{2}k(\|\mathbf{p}_i - \mathbf{p}_j\| - r_{ij})^2 \quad (2.9)$$

where:

- $E(\mathbf{p}_i, \mathbf{p}_j)$ is the elastic potential energy stored in the spring.
- k is the spring stiffness constant, which determines how strongly the spring resists deformation.
- $\mathbf{p}_i, \mathbf{p}_j$ are the positions of two connected mass points.

- r_{ij} is the rest length of the spring, i.e., the natural length when the spring is neither stretched nor compressed.

With these definitions in mind, we can now analyze how the potential energy behaves under different conditions (see Figure 2.4).

- When $\|\mathbf{p}_i - \mathbf{p}_j\| = r_{ij}$, the energy is **zero**, meaning the spring is at equilibrium.
- When $\|\mathbf{p}_i - \mathbf{p}_j\| > r_{ij}$, the spring is **stretched**, storing positive potential energy.
- When $\|\mathbf{p}_i - \mathbf{p}_j\| < r_{ij}$, the spring is **compressed**, also storing potential energy.

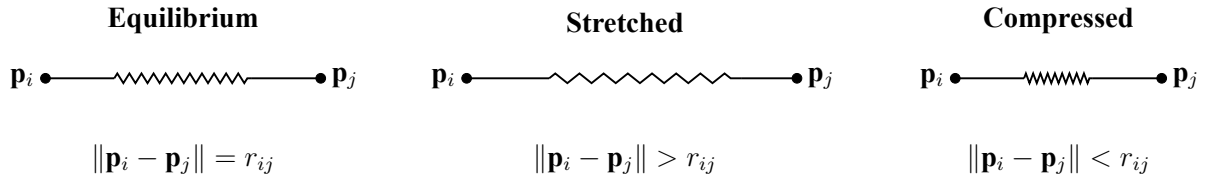


Figure 2.4: Three states of a mass-spring system: equilibrium, stretched, and compressed.

The $\|\mathbf{p}_i - \mathbf{p}_j\|$ term makes calculating E a non-linear problem. The Euclidean distance between two points is defined as in Equation (2.10):

$$\|\mathbf{p}_i - \mathbf{p}_j\| = \sqrt{(p_i^x - p_j^x)^2 + (p_i^y - p_j^y)^2 + (p_i^z - p_j^z)^2} \quad (2.10)$$

The square root in Equation (2.10) prevents the energy function from being a simple quadratic form. While standard approaches include gradient descent, the Gauss-Newton method, or Newton's method, these options are often too slow or computationally complex for real-time applications.

A more efficient optimization strategy involves reformulating the nonlinear energy function. For each spring, the problem is broken down into a smaller, independent optimization. This step requires finding a vector $\mathbf{d}_{ij}$ of length r_{ij} that remains as close as possible to the current spring vector (see Figure 2.5 and Equation (2.11)). The vector $\mathbf{p}_i - \mathbf{p}_j$ represents the displacement between two connected points in the system. The rest length r_{ij} is shown as the radius of the reference circle. The updated vector $\mathbf{d}_{ij}$ (in orange) is computed to satisfy the spring constraint while maintaining the structural integrity of the system.

$$(\|\mathbf{p}_i - \mathbf{p}_j\| - r_{ij})^2 = \min_{\mathbf{d}_{ij} \in \mathbb{R}^3, \|\mathbf{d}_{ij}\| = r_{ij}} \|(\mathbf{p}_i - \mathbf{p}_j) - \mathbf{d}_{ij}\|^2 \quad (2.11)$$

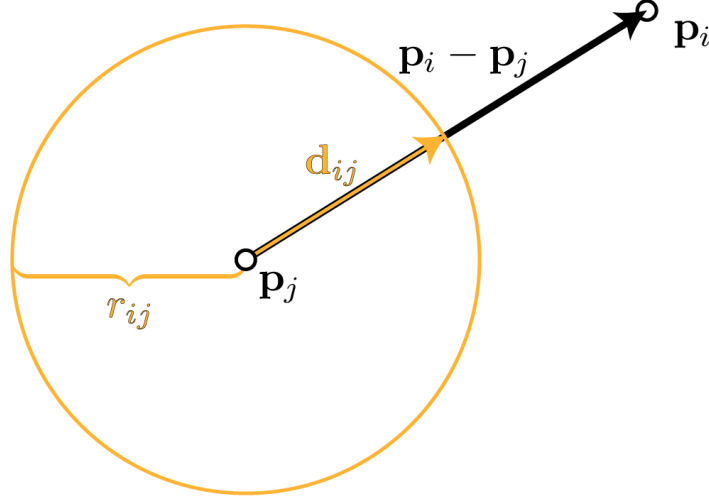


Figure 2.5: Geometric interpretation of the optimization process.

Having introduced the potential energy, we now revisit Equation (2.7) for further analysis. Since $\mathbf{f}(\mathbf{x})$ represents the force, and force is the negative gradient of potential energy [29], we have:

$$\mathbf{f}(\mathbf{x}) = -\nabla E(\mathbf{x}) \quad (2.12)$$

Substituting this into Equation (2.7) gives:

$$\mathbf{M}(\mathbf{x} - \mathbf{y}) = -h^2 \nabla E(\mathbf{x}) \quad (2.13)$$

This matches the critical point condition of the function:

$$g(\mathbf{x}) = \frac{1}{2}(\mathbf{x} - \mathbf{y})^T \mathbf{M}(\mathbf{x} - \mathbf{y}) + h^2 E(\mathbf{x}) \quad (2.14)$$

since setting $\nabla g(\mathbf{x}) = 0$ results in:

$$\mathbf{M}(\mathbf{x} - \mathbf{y}) + h^2 \nabla E(\mathbf{x}) = 0 \quad (2.15)$$

Thus, the solutions to Equation (2.7) correspond to the critical points of the function given in Equation (2.14). This leads to an optimization formulation $\min_{\mathbf{x}} g(\mathbf{x})$. This formulation, known as variational implicit Euler [30], allows solving the motion equations using optimization techniques instead of direct integration. This approach avoids nu-

merical instability and large time-step restrictions that often arise in explicit integration methods [31].

From the method described in the paper [25], we know that the potential energy of the system can be written as:

$$E(\mathbf{x}) = \min_{\mathbf{d} \in U} \left(\frac{1}{2} \mathbf{x}^T \mathbf{L} \mathbf{x} - \mathbf{x}^T \mathbf{J} \mathbf{d} + \mathbf{x}^T \mathbf{f}_{\text{ext}} \right) \quad (2.16)$$

where $U = \{(\mathbf{d}_1, \dots, \mathbf{d}_s) \in \mathbb{R}^{2s} \mid \|\mathbf{d}_i\| = r_i\}$ represents the set of rest-length spring directions.

Now, substituting Equation (2.16) into Equation (2.14), we obtain:

$$\begin{aligned} g(\mathbf{x}) &= \frac{1}{2} (\mathbf{x} - \mathbf{y})^T \mathbf{M} (\mathbf{x} - \mathbf{y}) + h^2 \min_{\mathbf{d} \in U} \left(\frac{1}{2} \mathbf{x}^T \mathbf{L} \mathbf{x} - \mathbf{x}^T \mathbf{J} \mathbf{d} + \mathbf{x}^T \mathbf{f}_{\text{ext}} \right) \\ &= \min_{\mathbf{x} \in \mathbb{R}^{3m}, \mathbf{d} \in U} \left[\frac{1}{2} \mathbf{x}^T \mathbf{M} \mathbf{x} - \mathbf{x}^T \mathbf{M} \mathbf{y} + \frac{1}{2} \mathbf{y}^T \mathbf{M} \mathbf{y} + h^2 \left(\frac{1}{2} \mathbf{x}^T \mathbf{L} \mathbf{x} - \mathbf{x}^T \mathbf{J} \mathbf{d} + \mathbf{x}^T \mathbf{f}_{\text{ext}} \right) \right] \end{aligned} \quad (2.17)$$

Since we are minimizing with respect to $\mathbf{x}$, the constant term $\mathbf{y}^T \mathbf{M} \mathbf{y}$ can be ignored. Also, for simplification we introduce $\mathbf{b}$ as $\mathbf{b} = h^2 \mathbf{f}_{\text{ext}} - \mathbf{M} \mathbf{y}$. We obtain the following:

$$g(\mathbf{x}) = \min_{\mathbf{x} \in \mathbb{R}^{3m}, \mathbf{d} \in U} \left(\frac{1}{2} \mathbf{x}^T (\mathbf{M} + h^2 \mathbf{L}) \mathbf{x} - h^2 \mathbf{x}^T \mathbf{J} \mathbf{d} + \mathbf{x}^T \mathbf{b} \right) \quad (2.18)$$

Thus, Equation (2.18) represents the final optimization problem, where the minimizer x^* provides the exact solution to the implicit Euler timestep.

2.4 Harmonic Weight Calculation

The theoretical foundation in this section references the Computer Graphics lectures from MIT Computer Science & Artificial Intelligence Laboratory¹ and the ACM SIGGRAPH 2014 course² to ensure academic accuracy.

Weights serve as numerical values assigned to each vertex on the model's surface mesh to determine the connection strength between the skin and the control system. This value represents the influence ratio of a specific bone or control point on a vertex whenever

¹https://15362.courses.cs.cmu.edu/fall2024content/lectures/12_lecture-11/12_lecture-11_slides.pdf

²<https://skinning.org/direct-methods-slides.pdf>

the character moves. To ensure geometric stability, weight sets are constrained between 0 and 1, and the sum of all weights at each vertex must always equal 1.

Specifically, if a vertex has a weight of 1.0 for a certain bone, it will follow every transformation of that bone perfectly. If the weight is only 0.5, the vertex will only move half the distance relative to the bone, with the remaining movement influenced by neighboring bones to create uniform deformation. In computer graphics software, these values are often visualized as a color gradient on the mesh surface, allowing users to easily observe and refine the influence zones of control handles, as shown in Figure 2.6.

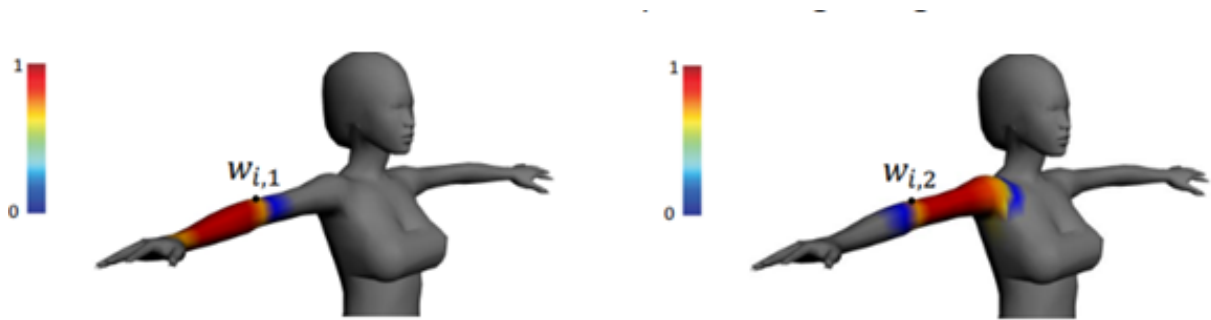


Figure 2.6: Visual representation of weights on the character's surface mesh.

Users can define weights manually or use automated calculation tools. In this research, the system applies the harmonic weights algorithm to establish these values automatically, a concept first introduced to character articulation by Joshi et al. from Pixar Animation Studios [32]. Rather than requiring the user to meticulously tune every vertex on the surface mesh, the algorithm distributes handle influence based on the character's own geometry. This significantly reduces processing time and ensures continuous variation at joints, helping the character maintain a natural shape during complex movements.

Technically, harmonic weights are determined by solving the Laplace equation over the character's geometric domain. Instead of calculating based on simple Euclidean distance, this algorithm forces weight values to propagate smoothly across the structural surface. Establishing a state of thermal equilibrium helps illustrate how the weights are distributed. Assigning a maximum value to one control joint and zero to others allows values to diffuse automatically across the surface. This diffusion results in a weight system with natural transitions that let the skin stretch without geometric kinking or overlapping during complex character motions.

To solve this continuous problem on a triangular mesh, the continuous Laplace equation $\Delta w = 0$ is discretized using the discrete Laplace-Beltrami operator. This reduction transforms the boundary-value problem into a system of linear equations (see Equation (2.19)):

$$\sum_{j \in N(i)} \omega_{ij}(w_i - w_j) = 0 \quad (2.19)$$

In this equation, $N(i)$ denotes the set of immediate neighboring vertices connected to vertex i , and ω_{ij} represents the geometric cotangent weights defined in Equation (2.20):

$$\omega_{ij} = \frac{1}{2}(\cot \alpha_{ij} + \cot \beta_{ij}) \quad (2.20)$$

where α_{ij} and β_{ij} are the two angles opposing the edge (i, j) in the adjacent triangles. The system is subject to Dirichlet boundary conditions, where a vertex hard-constrained to a selected control handle is assigned a value of 1, while vertices on all other handles are fixed to 0.

Solving this system determines the unknown weights for every unconstrained vertex on the mesh. Because the Laplace-Beltrami operator requires each free vertex to be the weighted harmonic average of its neighbors, the resulting values are distributed as smoothly as possible. Such a global balance eliminates abrupt value changes and produces natural, artifact-free deformations for the model.

2.5 Linear Blend Skinning Algorithm

The theoretical foundation in this section references the Computer Graphics lectures from Stanford University¹ and the ACM SIGGRAPH 2014 course² to ensure academic accuracy.

Linear Blend Skinning (LBS) is a standard technique widely used in the graphics industry to simulate surface mesh deformation according to skeletal movement (see Figure 2.7). LBS aims to eliminate surface tearing at joints by linearly interpolating vertex positions based on the influence of multiple bones. In this research, LBS is combined with

¹https://web.stanford.edu/class/cs248/pdf/class_13_skinning.pdf

²<https://skinning.org/direct-methods-slides.pdf>

the Harmonic weighting method described in Section 2.4 to produce a relatively continuous surface deformation, helping characters maintain more plausible shapes during interactive click-and-drag operations.

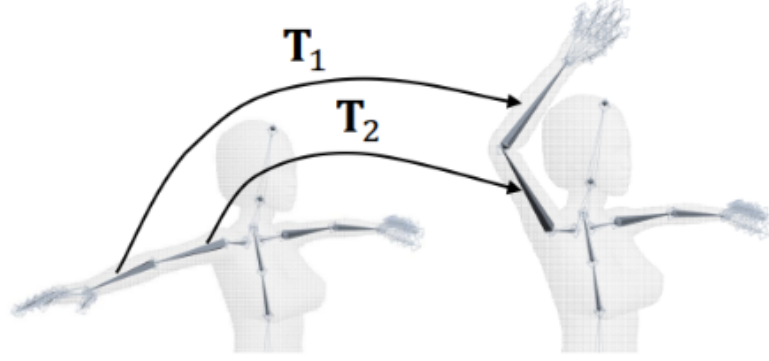


Figure 2.7: The Linear Blend Skinning process for mesh deformation.

In the mathematical representation of LBS (see Equation (2.21)), each mesh vertex v_j is not restricted to a single bone but is assigned to multiple bones with corresponding influence weights. The position of vertex v_j in world space is calculated based on a linear combination of transformations from the influencing bones:

$$v'_j = \sum_i w_{ij} T_i v_j \quad (2.21)$$

Where:

- v'_j is the deformed vertex coordinate in world space.
- v_j is the original vertex coordinate in model space (rest pose).
- T_i is the transformation matrix of the i -th bone, representing the translation and rotation from the rest pose to the current pose.
- w_{ij} is the influence weight of bone i on vertex j .

To ensure stable model deformation, the weights w_{ij} must satisfy the normalization condition $\sum_i w_{ij} = 1$ and must non-negative ($w_{ij} \geq 0$). While LBS offers superior computational performance, it still suffers from volume loss at joints when performing large-angle rotations.

Chapter 3

Proposed Methodology

3.1 System Overview

The core 3D modeling algorithm in this thesis is built upon the foundational Teddy system [2], while design insights from the MonsterMash framework [3] are integrated to enhance user interactivity. By supporting the merger of sketch domains directly in 2D space, the framework aims to offer a simplified workflow that attempts to make 3D modeling more casual, playful, and approachable, closely resembling the intuitive nature of traditional drawing. Figure 3.1 depicts the five primary stages of the transformation from a flat drawing to a 3D model.

First, the user creates a sketch on a 2D canvas. The system then decomposes these strokes into multiple regions and inflates each part using algorithms derived from the Teddy paper. Once individual meshes exist, the system stitches them together to establish a skeletal structure. Section 3.2 provides the details for mesh stitching and real-time rigging performed in steps two and three. Step four involves setting up a mass-spring system on the skeleton to facilitate dynamic secondary animation behaviors, as detailed in Section 3.3. Finally, Section 3.4 details the model editing and animation features implemented in step five to support interactive and flexible creation.

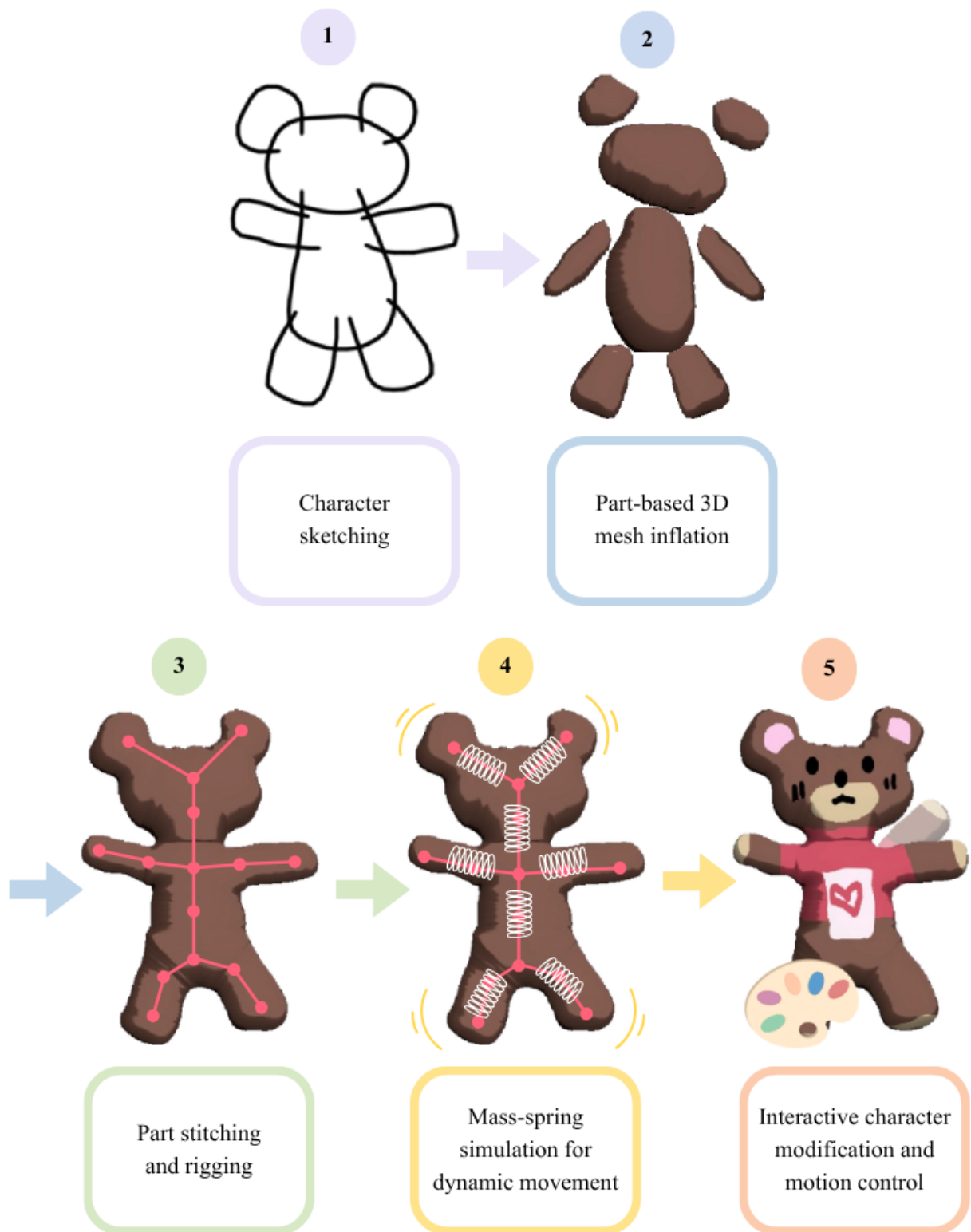


Figure 3.1: System overview workflow for constructing 3D models from sketches.

3.2 Mesh Stitching and Real-time Rigging

Stroke Segmentation into Regions After establishing the global boundary, the system analyzes individual strokes to determine their role within the character’s structure (see Figure 3.2a). At this stage, strokes are classified as either closed or open curves based on their geometric characteristics. Closed curves serve as base components and act as the foundation for the model. Open curves are calculated in subsequent steps to attach firmly to the main torso or previously drawn components.

After classification, the system automatically organizes these components into a hierarchical tree structure with the base component as the root, as illustrated in Figure 3.2b. Components directly attached to the base serve as its immediate child nodes, while subsequent parts are recursively linked as deeper descendants based on their attachment sequence.

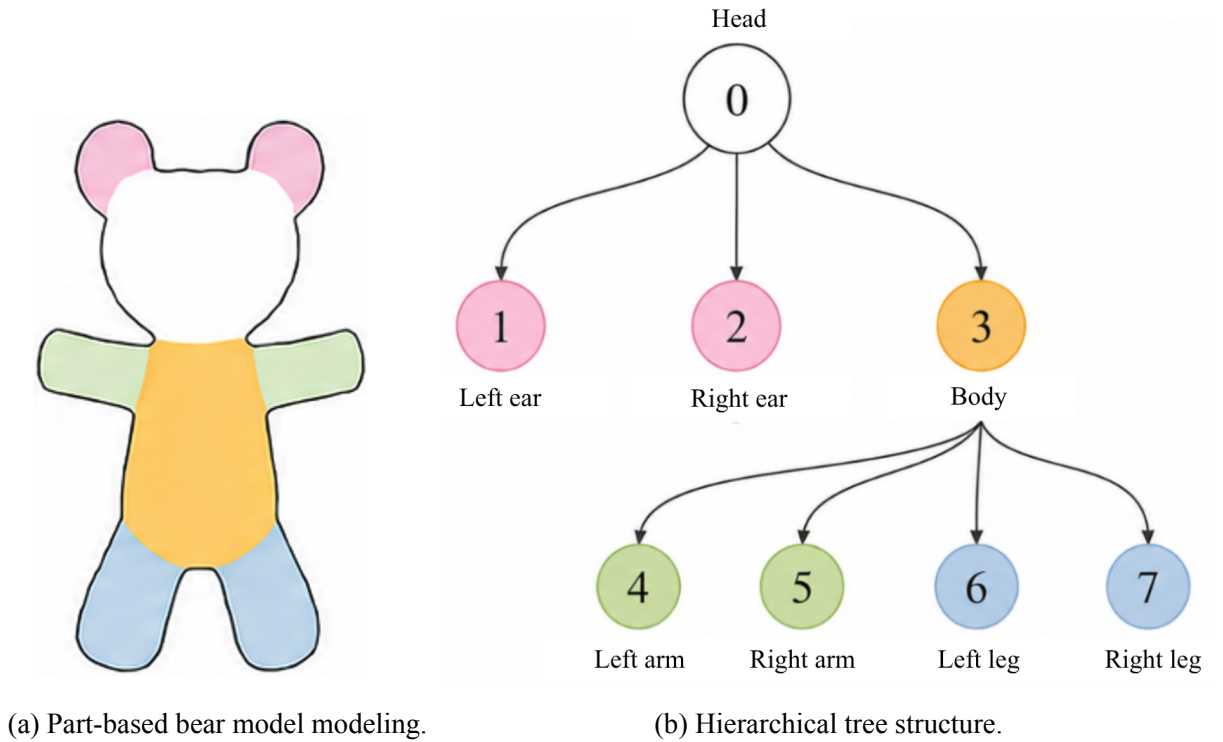


Figure 3.2: Hierarchical tree structure for managing the construction sequence and mesh stitching logic.

Mesh Domain Stitching To prepare for component integration, holes are made within the mesh domains at the exact points of contact. This step is achieved by splitting the surface along the connection boundary loop, denoted as β_p , which represents the closed

junction curve on the parent mesh. By duplicating the vertices along β_p , this operation creates zero-area openings that act as intermediate connection interfaces. As a result, the character's overall shape is preserved while providing the necessary foundation for accurately stitching limbs to the torso.

Following the hierarchy defined in the tree structure (see Figure 3.2b), the boundaries of auxiliary parts are connected to the corresponding hole edges on the parent mesh. This procedure executes a Breadth-First Search (BFS) algorithm to maintain a strict topological sequence, ensuring that added details are always accurately linked to their respective base structures. Once the topological connections are formed, a surface optimization step is performed locally around the newly stitched boundaries. Specifically, this process applies Laplacian smoothing constraints and vertex normal alignment across the junction loops to eliminate sharp geometric artifacts, thereby ensuring G^1 tangent continuity and preventing abnormal mesh distortions during subsequent deformations. The underlying logic for this operation is detailed in Algorithm 1.

Algorithm 1 Mesh Domain Stitching based on BFS Hierarchical Relationships

Require: Root component R_{base} , hierarchical tree structure T .

Ensure: The complete connected manifold mesh.

- 1: Initialize an empty queue Q
 - 2: Enqueue R_{base} onto Q
 - 3: **while** Q is not empty **do**
 - 4: Dequeue current parent component P_{parent} from Q
 - 5: Get the list of immediate child components $\{L_1, L_2, \dots, L_n\}$ of P_{parent} from T
 - 6: **for** each child component L_i **do**
 - 7: **Inter-domain Connection:**
 - 8: 1. Identify the boundary loop β_p on P_{parent} where L_i attaches.
 - 9: 2. Stitch the boundary edges of the child mesh L_i directly to the corresponding loop β_p on the parent mesh.
 - 10: Enqueue L_i onto Q
 - 11: **end for**
 - 12: **end while**
 - 13: **Surface Optimization:** Apply geometric constraints across all newly stitched boundaries to ensure smooth deformation transitions.
 - 14: **return** The fully integrated manifold mesh.
-

Real-time Rigging Upon the formation of the 3D model, a base skeletal structure is automatically determined using the central axis method from Teddy [2]. Instead of generating the entire skeleton at once, each domain is processed separately to optimize connectivity during part integration. Each component yields a distinct skeleton based on the central axis generated by the Teddy algorithm. For parts defined by open strokes, a temporary closed loop is established to define the spatial volume before initializing the bones.

The global skeleton initializes using the skeleton of the root component, which is the part at index 0 in the hierarchy illustrated in Figure 3.2. To link these structures, the midpoint of the contact line between the parent and child components is calculated. The algorithm then identifies the nearest joints on both the parent and child skeletons. A new bone segment is then created to act as a direct bridge between them. Once connected, all individual segments are integrated into a single hierarchical tree structure. Finally, skinning weights are computed across the entire stitched mesh to ensure that the skin at intersection points remains intact during movement. The logic for this hierarchical setup is presented in Algorithm 2, and the weights are subsequently calculated using the harmonic skinning method described in Section 2.4.

Algorithm 2 Hierarchical Skeleton Connection Setup based on BFS

Require: Root component R_{base} , hierarchical tree structure T .

Ensure: The fully connected global skeleton $\mathbf{G}$.

- 1: Extract individual skeleton S_{base} from the geometric shape of R_{base}
 - 2: Initialize the global skeleton $\mathbf{G} \leftarrow S_{base}$
 - 3: Initialize an empty queue Q
 - 4: Enqueue R_{base} onto Q
 - 5: **while** Q is not empty **do**
 - 6: Dequeue current parent component P_{parent} from Q
 - 7: Get the list of immediate child components $\{L_1, L_2, \dots, L_n\}$ of P_{parent} from T
 - 8: **for** each child component L_i **do**
 - 9: Extract individual skeleton S_i based on the geometric shape of L_i
 - 10: **Hierarchical Linkage:**
 - 11: 1. Identify the connection boundary β_p on P_{parent} where L_i attaches.
 - 12: 2. Find the nearest joint $\mathbf{j}_{parent}$ on the existing global skeleton $\mathbf{G}$ within the region of β_p .
 - 13: 3. Find the root joint $\mathbf{j}_{child}$ of the child skeleton S_i .
 - 14: 4. Create a connecting edge (new bone) between $\mathbf{j}_{parent}$ and $\mathbf{j}_{child}$.
 - 15: Merge the graph of S_i into the global skeleton $\mathbf{G}$
 - 16: Enqueue L_i onto Q
 - 17: **end for**
 - 18: **end while**
 - 19: **return** The complete hierarchical global skeleton $\mathbf{G}$.
-

3.3 Integrating Mass-Spring Systems for Dynamic Secondary Motion

Thanks to the work by Liu et al. [25], computing the mass-spring system state at each time step is simplified into a straightforward iterative process based on a local-global approach:

- **Step 1 (Local Update):** Given the current positions $\mathbf{p}$, determine d_{ij} for each spring.
- **Step 2 (Global Update):** Using all computed vectors d_{ij} , update $\mathbf{x}$ by minimizing the quadratic energy function described in Equation (2.18).

- **Step 3 (Convergence Check):** If the solution has not yet converged, return to Step 1. If the maximum number of iterations is exceeded, terminate the algorithm.

3.3.1 Building Matrices

Before solving for the new positions of nodes, three key matrices $\mathbf{L}$, $\mathbf{J}$, and $\mathbf{M}$ are constructed based on the original framework [25]. Specifically, $\mathbf{M} \in \mathbb{R}^{3n \times 3n}$ is a diagonal mass matrix containing the nodal masses across all three dimensions. $\mathbf{L} \in \mathbb{R}^{3n \times 3n}$ represents the symmetric, sparse Laplacian matrix encoding the spring network connectivity and stiffness weights k_k . $\mathbf{J} \in \mathbb{R}^{3n \times 3s}$ acts as a directed incidence matrix that maps local spring deformations back onto the global node coordinates. The construction process for these matrices is detailed in Algorithm 3.

Algorithm 3 Construction of $\mathbf{L}$, $\mathbf{J}$, and $\mathbf{M}$

Require: Spring system with n points and s springs.

- 1: Initialize empty sparse matrices $\mathbf{L}$, $\mathbf{J}$, and $\mathbf{M}$.
 - 2: $h^2 \leftarrow$ time step squared.
 - 3: **for each** spring k connecting nodes (i, j) **do**
 - 4: **for each** coordinate $d \in \{x, y, z\}$ **do**
 - 5: $l_{3i+d, 3i+d} += k_k, l_{3j+d, 3j+d} += k_k$
 - 6: $l_{3i+d, 3j+d} -= k_k, l_{3j+d, 3i+d} -= k_k$
 - 7: **end for**
 - 8: **end for**
 - 9: **for each** spring k connecting nodes (i, j) **do**
 - 10: **for each** coordinate $d \in \{x, y, z\}$ **do**
 - 11: $j_{3i+d, 3k+d} += k_k, j_{3j+d, 3k+d} -= k_k$
 - 12: **end for**
 - 13: **end for**
 - 14: **for each** node i **do**
 - 15: **for each** coordinate $d \in \{x, y, z\}$ **do**
 - 16: $m_{3i+d, 3i+d} \leftarrow m_i$
 - 17: **end for**
 - 18: **end for**
 - 19: Precompute $\mathbf{A} = \mathbf{M} + h^2 \mathbf{L}$ and factorize for solving.
-

3.3.2 Local Update

The local update relies on local information for each spring. Specifically, it computes the spring direction vectors for each spring in the system, ensuring that they maintain their rest length constraint. Algorithm 4 details the local update step.

Algorithm 4 Local Update for Mass-spring System

Require: Current positions $\mathbf{p}$, rest lengths r_{ij} of springs

Ensure: Updated spring direction vectors $\mathbf{d}_{ij}$

- 1: $j \leftarrow 0$
 - 2: **for** each spring $(i, j) \in \text{system's spring list}$ **do**
 - 3: Compute displacement vector: $\mathbf{p}_{12} \leftarrow \mathbf{p}_i - \mathbf{p}_j$
 - 4: Normalize the displacement: $\hat{\mathbf{p}}_{12} \leftarrow \frac{\mathbf{p}_{12}}{\|\mathbf{p}_{12}\|}$
 - 5: Compute updated spring direction: $\mathbf{d}_{ij} \leftarrow r_{ij} \cdot \hat{\mathbf{p}}_{12}$
 - 6: Store $\mathbf{d}_{ij}$ in spring direction list
 - 7: $j \leftarrow j + 1$
 - 8: **end for**
-

3.3.3 Global Update

The global update, on the other hand, involves all springs simultaneously. In this step, the node positions $\mathbf{x}$ are solved while keeping the local deformations $\mathbf{d}$ fixed. Starting from the quadratic minimization problem described in Equation (2.18), the optimal $\mathbf{x}$ is obtained by taking the gradient with respect to $\mathbf{x}$ and setting it to zero, as shown in Equation (3.1):

$$\frac{\partial}{\partial \mathbf{x}} \left(\frac{1}{2} \mathbf{x}^T (\mathbf{M} + h^2 \mathbf{L}) \mathbf{x} - h^2 \mathbf{x}^T \mathbf{J} \mathbf{d} + \mathbf{x}^T \mathbf{b} \right) = 0 \quad (3.1)$$

Since $\mathbf{L}$ is symmetric and positive semi-definite, the system matrix $\mathbf{M} + h^2 \mathbf{L}$ is symmetric positive definite. Consequently, the derivative from Equation (3.1) simplifies to Equation (3.2):

$$(\mathbf{M} + h^2 \mathbf{L}) \mathbf{x} - h^2 \mathbf{J} \mathbf{d} + \mathbf{b} = 0 \quad (3.2)$$

Rearranging the terms in Equation (3.2) yields the final global linear system in Equation (3.3):

$$(\mathbf{M} + h^2\mathbf{L})\mathbf{x} = h^2\mathbf{J}\mathbf{d} - \mathbf{b} \quad (3.3)$$

This is a convex quadratic system where $\mathbf{M}$ represents the mass matrix, $h^2\mathbf{L}$ accounts for stiffness from the implicit Euler integration, $h^2\mathbf{J}\mathbf{d}$ encodes local deformation constraints, and $\mathbf{b}$ contains external forces and previous state dependencies.

Since the system matrix $\mathbf{M} + h^2\mathbf{L}$ is symmetric positive definite, this guarantees the existence of a unique solution, which we solve efficiently using Cholesky factorization. By precomputing this factorization (since the system matrix remains constant when mass, timestep, and connectivity do not change), we significantly speed up the computation in each iteration. Algorithm 5 details the global update step.

Algorithm 5 Global Update Step

Require: Precomputed matrices $\mathbf{M}$, $\mathbf{L}$, $\mathbf{J}$ and external forces $\mathbf{f}_{\text{ext}}$.

1: Compute the right-hand side:

$$\mathbf{b} = h^2\mathbf{f}_{\text{ext}} - \mathbf{M}\mathbf{y}.$$

2: Compute $\mathbf{x}$:

$$\mathbf{x} = (\mathbf{M} + h^2\mathbf{L})^{-1}(h^2\mathbf{J}\mathbf{d} - \mathbf{b}).$$

3: Update the system state with $\mathbf{x}$.

3.3.4 Convergence Check

For ease of implementation, our simulation assumes convergence after a fixed number of iterations (e.g., 100). However, more advanced stopping criteria could be employed to adaptively determine convergence based on error thresholds or energy minimization.

Algorithm 6 Mass-Spring Solver with Fixed Iterations

Require: Maximum iterations n

1: Store previous state: $\mathbf{p}_{n-1} \leftarrow \mathbf{p}_n$

2: **for** $i = 1$ to n **do**

3: **Local Step:** Update spring directions $\mathbf{d}_{ij}$ by normalizing displacement vectors

4: **Global Step:** Solve for $\mathbf{x}$ using the precomputed Cholesky factorization

5: **end for**

3.4 Improving User Interaction

In developing this system, significant emphasis was placed on optimizing the interaction experience to foster and maintain the user’s creative flow. This responsive framework is what translates the underlying algorithms into tangible, practical value. Without an intuitive interface that allows artists to leverage these technical tools for their creative output, even the most advanced mathematical models risk remaining mere theoretical concepts with limited real-world utility.

The system is built upon three core criteria:

- *Immediate Feedback:* Every stroke or modification displays results instantly, allowing users to grasp the current state of the model.
- *Intuitive and natural control:* The software operations align with traditional drawing habits, ensuring predictable outcomes and reducing the learning curve for users during part integration.
- *Flexible prototyping and experimentation:* The system allows users to generate models quickly and independently adjust physics parameters for individual components. Alternative designs or parameter modifications can be tested easily without the burden of a time-consuming restart.

The system provides five primary operational functions: **(i)** a sketching mode that transforms hand-drawn strokes into 3D models and skeletons as described in Figure 3.1; **(ii)** drawing directly on the 3D model surface; **(iii)** internal skeletal structure adjustment by removing redundant joints to simplify character control; **(iv)** a regional selection tool for bones to set varying stiffness k and damping k_d for mass-spring systems, creating different physical properties for different parts; and **(v)** an animation mode to produce short motion sequences for the character.

3.4.1 Painting on Model Surfaces

The painting algorithm is derived from the Teddy system [2] with adjustments to suit the Unity environment and the integrated programming logic of the system.

The first step involves hit detection. When a user clicks or touches the screen, the system casts an invisible ray from the camera through the contact point into 3D space. The

first point of intersection between this ray and the character's surface defines the actual collision coordinates.

The next step maps the resulting 3D position back to the image space. Since these models are generated procedurally, they lack pre-existing texture coordinates. A planar projection is utilized to convert the 3D collision point into 2D UV space. This coordinate mapping allows the system to identify exactly which pixel on the texture map corresponds to the touched area on the character's body. At this location, the system modifies the pixel colors.

Finally, during the display update phase, only the texture map on the GPU is updated rather than creating an entirely new material for the character. This approach significantly optimizes memory usage and ensures that painting on one character does not inadvertently affect the appearance of other objects within the scene.

3.4.2 Manual Skeleton Adjustment

Because Teddy's algorithm constructs the skeletal structure based on mesh density [2], the resulting number of joints can be excessively large, resulting in a preliminary layout that is not fully optimized for the user's intent. Therefore, once the skeleton is initialized, users can remove redundant joints to simplify character control. However, since this skeleton is derived directly from the underlying geometry, only the deletion of unnecessary joints is permitted, rather than their relocation or free insertion. The original topology of the model is thus preserved through this constraint.

3.4.3 Regional Control Mechanism

The joint selection feature allows users to experiment with unique physical properties for specific areas of the character's body. By using the selection tool, users can isolate any group of joints to assign tailored simulation parameters.

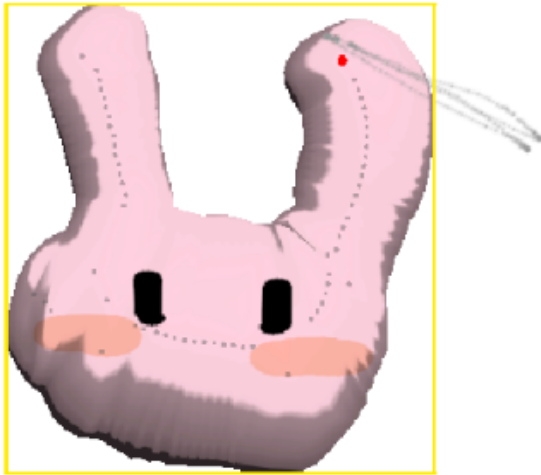
All joints within the selected region possess the capability to interact physically with gravity or forces exerted by drag-and-drop actions. Joints located outside the selection automatically switch to a fixed state. These joints act as a solid support frame, helping the character maintain a stable pose and preventing the model from collapsing while other parts are in motion.

Users can customize the stiffness constant k or the damping coefficient k_d for each designated region. For instance, the torso can be assigned high stiffness to maintain its structural integrity under external forces, whereas peripheral features like ears or tails can be configured with lower values to exhibit soft elasticity. All localized parameters can be reset or modified instantly, allowing users to experiment with various interaction scenarios without altering the base skeletal structure.

3.4.4 Animation Mode

The animation mode provides tools to generate motion sequences for the completed character model.

Instead of adjusting complex parameters, users can create animations or physical interactions simply by dragging the mouse directly on the screen. The user selects one or more joints as control points. When these points are dragged, the system automatically records their path as a movement trajectory (see Figure 3.3). This process is similar to drawing a path for the character to follow. The movement of the control point is visually displayed as a curve on the screen. Upon mouse release, the recorded trajectory enters an automatic loop to simulate simple and repetitive motions such as nodding, waving, or jumping.



(a) The user selects a joint as a control point and drags the mouse to draw a trajectory line.



(b) The control point moves along the trajectory line after drawing

Figure 3.3: Motion generation via trajectory sketching

Chapter 4

Results and Discussion

4.1 System Implementation and Experimental Setup

The system is developed on the Unity platform (version 6000.4.0f1) using the C# language (.NET 8), utilizing the Universal Render Pipeline (version 17.4.0). Figure 4.1 illustrates the graphical user interface of the developed framework, with detailed functional components explained in the subsequent sections. Additionally, the framework can export models in GLB file format.

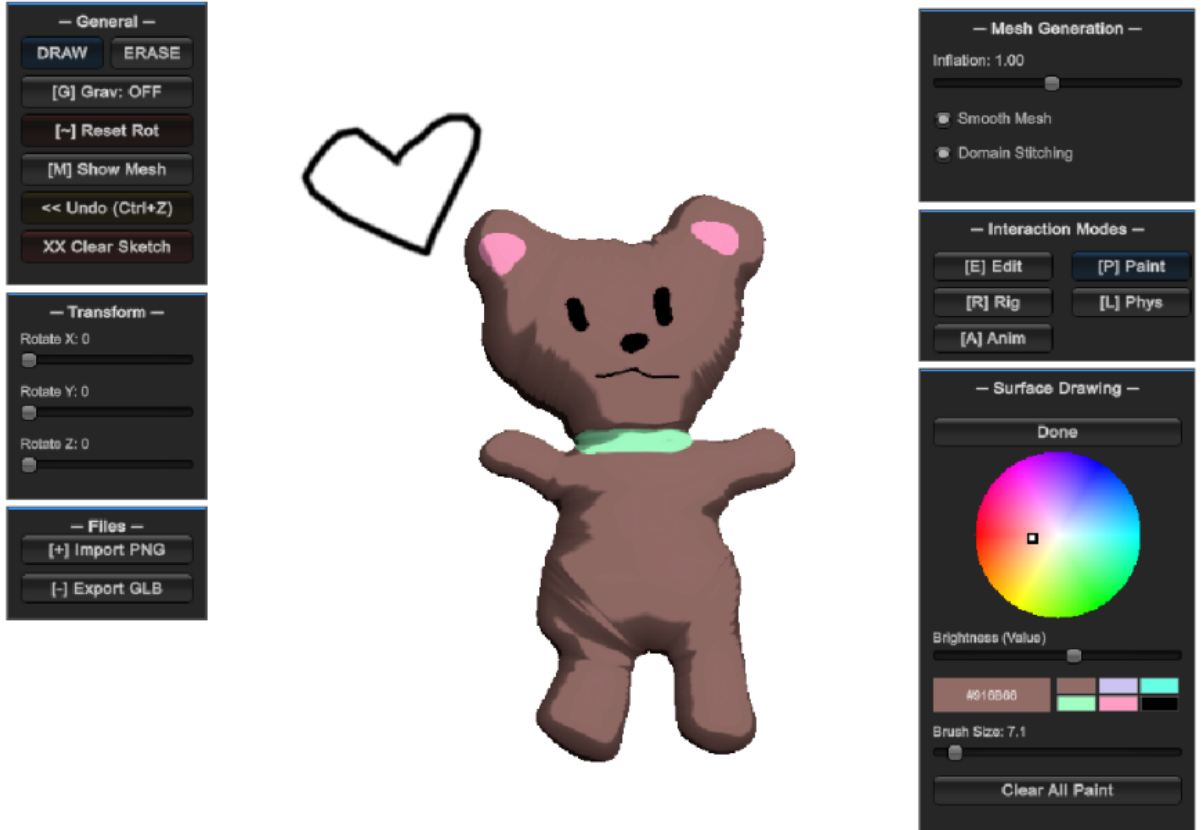


Figure 4.1: The graphical user interface of the developed framework.

Regarding the algorithmic implementation, the thesis uses the reference source code from Masatatsu Nakamura's GitHub (<https://github.com/mattatz/unity-teddy>) as a starting point. This version includes the core principles from the research by Igarashi et al. [2], such as mesh triangulation and basic geometric inflation.

Building upon this foundation, advanced geometric processing algorithms and new interactive control systems were developed to meet the specific requirements of this thesis. Following the sub-stepping strategy [33], the time step is set to $h = 1/60$ seconds, corresponding to a 60 FPS simulation rate. As for external forces, the simulation considers only gravitational acceleration $g = 9.8 \text{ m/s}^2$ [29] and interactive external forces induced by direct user manipulation, such as dragging.

For performance evaluation, all experiments were conducted on an AMD Ryzen 7 7840H processor with Radeon 780M Graphics. To ensure consistency, auto-parallelization was disabled, and all computations were performed on a single thread.

The complete source code for the project implementation is accessible at: <https://github.com/letmaplespillthetea/teddy-thesis>.

4.2 Visual Results of Generated Models

Figure 4.2 showcases several creative models produced by talented artists utilizing the proposed system. A visual comparison of models created by the original source by Masatatsu Nakamura, our proposed system, Teddy, and MonsterMash is summarized in Table 4.1.

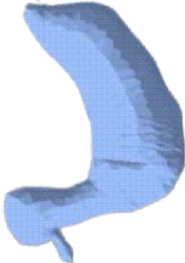


Figure 4.2: Some models created by users from the system

The original source code served as a basic implementation of the Teddy algorithm for individual shapes. Building upon this, the current research developed the ability to connect multiple sketch domains into a single unified entity. The new system allows users to paint directly onto the mesh surface to create more vibrant models. Furthermore, the integration of a skeletal rig combined with a mass-spring system enables characters to perform simple motions such as waving, nodding, or jumping. However, compared to the results from the top-tier SIGGRAPH papers, the system still faces limitations regarding spatial depth.

Specifically, the Teddy algorithm supports sketching directly in 3D space. It allows users to rotate to the back of a character to add details like a tail. In contrast, this student-

Table 4.1: Comparison of model results from the original source code, student-developed models, Teddy, and MonsterMash.

System	Example 1	Example 2	Example 3
Original Source			
Proposed System			
Teddy			
MonsterMash			

developed system is currently limited to 2D plane construction and cannot generate additional volumes on the backside of an existing model.

When compared to MonsterMash, although both systems approach modeling from a 2D space, this implementation does not yet handle stroke layer ordering. MonsterMash can explicitly define whether a leg is in front of or behind the torso, whereas the current system defaults all volumes to the same depth. Consequently, while MonsterMash only requires 2D input, it can still produce volumes located at the back. Nevertheless, the proposed system possesses unique creative advantages. Instead of requiring pre-existing images for surface textures like MonsterMash, users can draw and color directly on the interface, offering a natural experience similar to painting a physical statue. Regarding animation, this research utilizes a skeletal approach integrated with a mass-spring system instead of pure mesh deformation. This mechanism allows for smooth secondary motion, a feature not supported by the original Teddy system.

Table 4.2 presents a comparative analysis among the Teddy source code by Masatatsu Nakamura, the original Teddy system, MonsterMash, and the framework developed in this research. Although the current results have limitations and are not yet as widely applicable as the professional systems, the opportunity to learn from these two breakthrough works in sketch-based 3D modeling has been invaluable for the student to learn how to bridge the gap between 2D artistic intent and 3D geometric representation.

Table 4.2: Feature comparison between the proposed system and related research

Comparison Aspect	Source Code	Teddy	Monster Mash	Proposed System
3D Model Generation	✓	✓	✓	✓
Direct Painting on Model	✗	✓	✗	✓
Image-based Texturing	✗	✗	✓	✓
Primary Motion	✗	✗	✓	✓
Arbitrary Volume Connection	✗	✓	✓	✗
Real-time Skeletonization	✗	✗	✗	✓
Secondary Motion	✗	✗	✗	✓

4.3 Computational Performance Analysis

Table 4.3 provides a detailed breakdown of model complexity and the corresponding execution metrics across different operational phases, spanning from the initial offline sketch triangulation to real-time rigging and runtime skeleton updates. It is worth noting that the heavy initialization process operates within a range of a few seconds. For instance, the most complex model “Tree” (see Figure 4.2e) containing 3,566 vertices and 160 joints requires 5.999 seconds to complete its topology generation. This geometric construction time, along with the resulting mesh density, is inherently proportional to the stroke scale and length on the canvas. Larger and more intricate sketch dimensions scale up the number of generated vertices, triangles, and joints during the triangulation and medial axis extraction phases, consequently increasing the initial computational overhead.

Table 4.3: Model Complexity and Performance Statistics Across Phases

Name	Vertices (V)	Joints (n)	Initialization	Runtime Update
			Time (sec) $\mathcal{O}(V^2 + n^3)$	Time (ms) $\mathcal{O}(I \times n)$
Whale (Fig. 4.2a)	2301	113	3.028	< 0.1
Rabbit head (Fig. 4.2b)	1518	76	1.432	< 0.1
Star (Fig. 4.2c)	1577	72	1.471	< 0.1
Cat (Fig. 4.2d)	3106	145	4.922	< 0.1
Tree (Fig. 4.2e)	3566	160	5.999	< 0.1
Teddy (Fig. 4.2f)	2911	135	4.421	< 0.1

However, this one-time generation latency does not impede the *Immediate Feedback* requirement during actual user interaction. Because the mass, timestep, and structural connectivity of the spring system remain constant after creation, the computationally expensive sparse Cholesky factorization of the system matrix $A = M + h^2 L$ is performed **only once** during this initialization phase.

Consequently, when the user enters the animation mode, the local-global solver bypasses matrix factorization entirely. At each subsequent frame, the system only needs to perform forward and backward substitutions to solve the pair of triangular systems formulated in

Equation 4.1:

$$Ly = b, \quad \text{and} \quad L^T x = y \quad (4.1)$$

Due to the tree-like hierarchical structure of the skeleton, the Laplacian matrix L is highly sparse, restricting the number of non-zero elements per row to a small constant. This structural sparsity drops the computational complexity of the global update step to a linear relation $\mathcal{O}(I \times n)$ per frame, where n is the number of joints and I is the number of local-global iterations.

In this implementation, even with a maximum allocation of $I = 100$ iterations to ensure tight convergence, the runtime execution for the heaviest test case ($n = 160$) requires less than 0.1 milliseconds of CPU time (see Table 4.3). This represents a negligible fraction of the 16.67 milliseconds budget required for a standard 60 FPS display ($h = 1/60$ seconds). Therefore, by offloading the $\mathcal{O}(n^3)$ matrix decomposition to the initialization phase and keeping the runtime complexity strictly linear ($\mathcal{O}(n)$), the system sustains a stable 60 FPS simulation, delivering real-time, interactive responses to user drag-and-drop inputs.

4.4 Rigging and Dynamic Motion Analysis

4.4.1 Motion Comparison

Comparative tests were conducted on the same model to evaluate the visual and physical effects of the mass-spring integration (see Figure 4.3).

Without the mass-spring system, the character operates strictly as a single rigid body. When a rotation or movement is applied, all parts of the mesh move instantly and uniformly according to the skeletal joints. As a result, peripheral features like the rabbit ears remain completely stiff, lacking any natural delay or flexible deformation relative to the character's body.

Conversely, when the mass-spring system is integrated into the skeleton, these joints gain the ability to react to physical forces. When the user manipulates the model via drag-and-drop actions, gravity and momentum pull the mass points away from their original rest positions. The system then calculates these dynamic displacements frame-by-frame using the solver detailed in Equation 4.1.

By applying these updated joint positions to the surface mesh through harmonic blend skinning, the system automatically generates secondary motions. This approach attempts

to replicate the classic animation principles of *follow-through* [4] in a procedural manner. As a result, peripheral features such as ears or tails can exhibit visible delay and elastic rebounds. This physics-based integration helps the generated characters appear softer and more dynamic during interaction, reducing the need for manual keyframing.

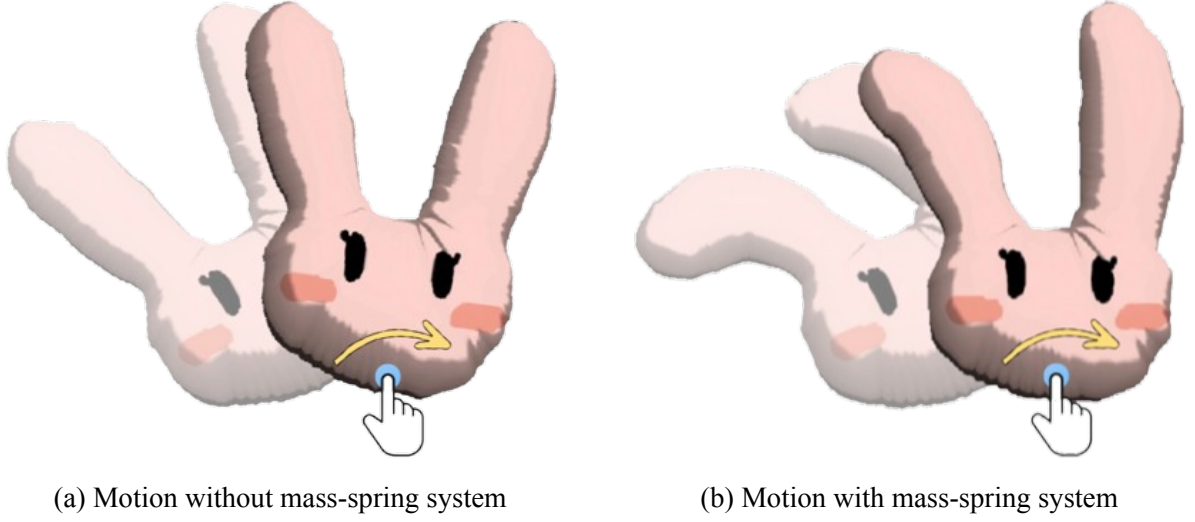


Figure 4.3: Comparison of system performance with and without a mass-spring system on the same model.

4.4.2 Effects of Parameters

In physics-based simulations, the dynamic behavior of the mass-spring network is governed by two key structural parameters: the spring stiffness constant k , measured in Newtons per meter (N/m), and the damping coefficient k_d , measured in Newton-seconds per meter (N · s/m) [29]. Within the developed framework, these parameters are normalized to a dimensionless range between 0 and 1 for intuitive configuration. While stiffness dictates the magnitude of the restoring force and resists deformation, damping controls the rate of energy dissipation to prevent endless oscillations. Users can adjust both k and k_d through the interface to define different material properties. For instance, setting high stiffness and damping values preserves the shape of rigid components like the torso, whereas lowering these parameters allows flexible areas, such as ears or tails, to exhibit softer, compliant elastic rebounds.

Experimental results demonstrate that adjusting the physical parameters is critical to balancing dynamic realism with visual stability. The influence of various parameter combinations on the model’s trajectory is illustrated in Figure 4.4, with a detailed frame-by-

frame displacement plot of a single point on the rabbit ear provided in Figure 4.5.

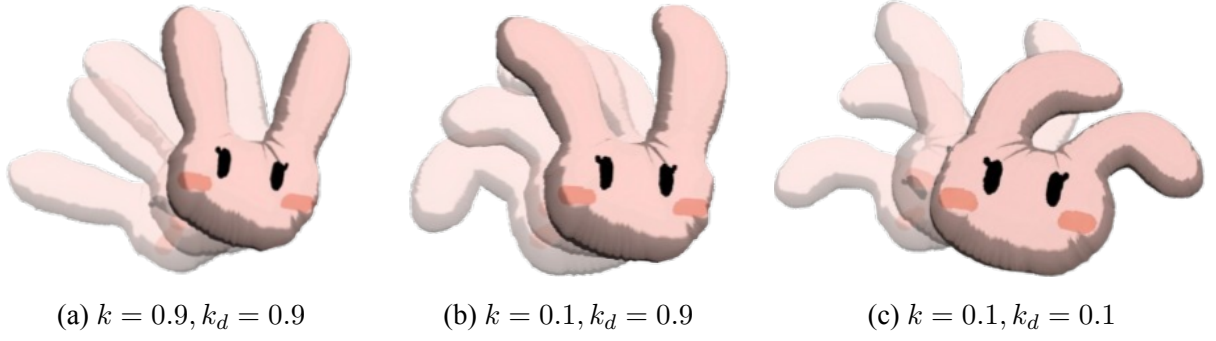


Figure 4.4: Comparison of the influence of spring stiffness k and damping coefficient k_d on model motion effects.

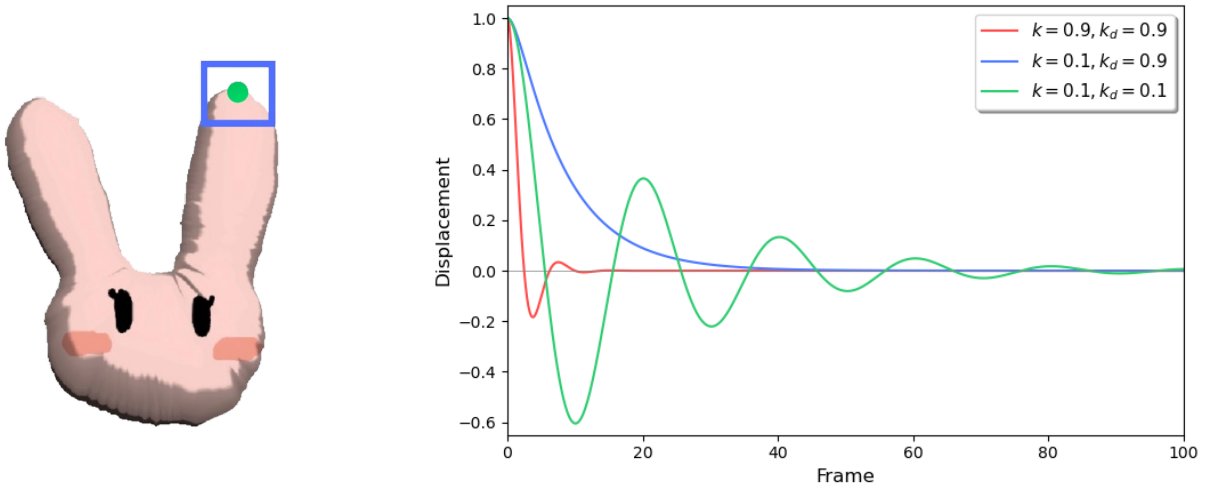


Figure 4.5: Marked point on the rabbit ear (left) and a plot showing the displacement of that point when adjusting different physical parameters (right).

When both stiffness and damping are set to high values ($k = 0.9, k_d = 0.9$), the system responds with high intensity and speed. As shown by the red curve in Figure 4.5, the displacement drops sharply within the first ten frames, exhibiting a single brief overshoot past the equilibrium line before the oscillation is immediately suppressed. Consequently, the model behaves almost like a rigid body, snapping back swiftly to its rest pose.

Lowering the stiffness to $k = 0.1$ while maintaining a high damping coefficient ($k_d = 0.9$) results in an overdamped behavior. The blue curve indicates a smooth, gradual asymptotic decrease in displacement that stretches well past the 40-frame mark before stabilizing. Visually, such a configuration generates a heavy, sluggish motion where the ears

lag significantly behind the body’s movement.

Conversely, when both parameters are assigned low values ($k = 0.1, k_d = 0.1$), the underdamped system allows the kinetic energy to dissipate slowly. The green curve clearly shows a classic decaying sinusoidal oscillation, where the ear point swings past the equilibrium line multiple times before finally stabilizing around frame 80. Such a parameter mix yields a loose, highly elastic rebound effect.

These observations demonstrate that tuning the relationship between k and k_d enables the framework to systematically approximate different material behaviors. By adjusting these values, the character’s movement can range from stiff and responsive to soft and jiggly, all while maintaining low computational overhead.

The main effects of stiffness (k) and damping (k_d) on the animation are summarized in Table 4.4.

Table 4.4: Effects of Stiffness (k) and Damping (k_d) on Character Motion

Parameter	Setting	Motion Characteristics
Stiffness (k)	High	Rigid, returns to original shape quickly, less deformation
	Low	Soft, flexible, large deformations, highly elastic
Damping (k_d)	High	Heavy drag, animation stops quickly, sluggish movement
	Low	Loose, jittery, over-oscillating, animation lasts too long

4.5 User Feedback

To gain a more comprehensive and objective overview, an informal study was conducted to determine the practicality and interactivity of the system through a small survey and in-depth interviews throughout the research process. The target group was selected to represent various levels of technological proficiency. This group included three formally trained artists: a student at the University of Industrial Fine Arts, a graduate from the same institution, and a graduate student at the Central Academy of Fine Arts in China. All three artists are proficient in 2D graphics software; two are experienced with 3D tools, while the third possesses a foundational understanding of the field. This group was contrasted with two engineering students from the University of Engineering and Technology who identified as having no artistic talent or experience with professional

art tools, alongside an eight-year-old child.

The task for the users was to build a short 3D scene in their personal style from the initial steps. The actual models they created are detailed in Figure 4.2. Following the task, in-depth interviews were conducted to record qualitative assessments of their practical experience with the system. Additionally, to provide an objective quantitative foundation, a survey utilizing a 5-point Likert scale (see Appendix A) was implemented. The evaluation items were adjusted and contextualized to match the practical workflow of the system, rather than strictly following the theoretical criteria. This survey assessed the three main dimensions outlined in Section 3.4: (1) ease of use and naturalness, (2) comfort during experimentation and implementation of ideas, and (3) overall satisfaction level. The results are illustrated in Figure 4.6.

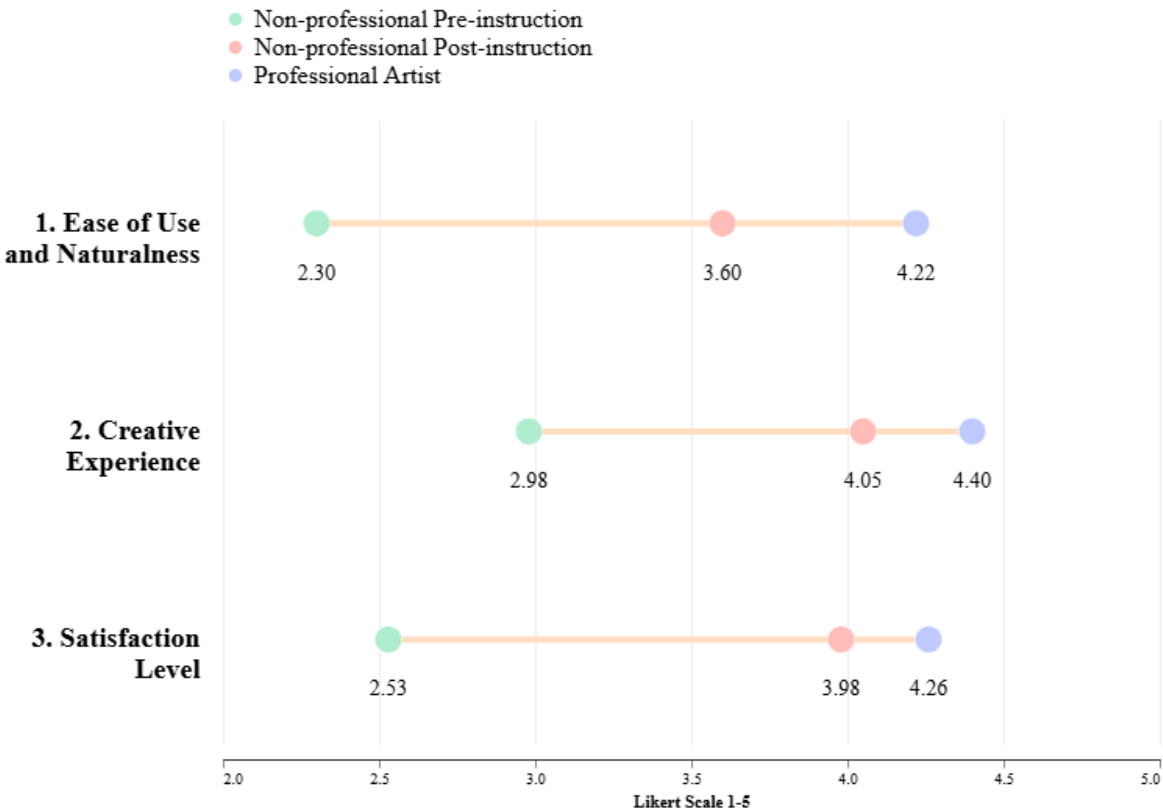


Figure 4.6: Survey results evaluating user experience across three core criteria.

Both professional artists and non-professional users expressed interest in interacting with the system and highly valued the utility of instantly converting hand-drawn characters into 3D models.

The survey results revealed a clear differentiation in system accessibility between the

groups. For the artists with experience in professional graphics software, there were few obstacles in interacting with the interface. Their ease-of-use rating averaged 4.22/5.0 after only a preliminary walkthrough. Conversely, the non-professional group initially exhibited noticeable confusion, with early experience scores averaging 2.30/5.0. However, after specific guidance and detailed explanations of the operations, this figure increased significantly to 3.60/5.0. This situation was anticipated. Based on the researcher's observations and feedback from professional artists, graphics software generally features complex interfaces that can be overwhelming for beginners.

It is evident that although the models generated by the system have certain limitations as analyzed in Table 4.2, the artists were still able to produce impressive results. One artist even remarked that the system serves as an excellent exercise for fostering creativity within specific constraints. Because the models are generated directly from strokes, they do not possess the polished appearance found in major studies like Teddy or MonsterMash. However, this characteristic helps preserve the spirit of the original sketch and creates a unique aesthetic style. According to the artist, this is a crucial factor in the development of initial experimental prototypes.

Regarding the creative experience, the most impressive aspect of the system is the immediate feedback. This allows users to see instant results from their actions and fosters a natural, uninterrupted flow of inspiration. A significant advantage lies in the system's ability to automatically calculate secondary motions such as gravitational fall and bounce. These features make characters appear more vibrant and realistic without requiring the user to manually edit complex individual frames. Furthermore, the interest is heightened by the ability to flexibly control the stiffness and softness of different character parts, providing a sense of total mastery over the creative entity.

A deeper quantitative analysis shows that interface handling skills directly influence the quality of the experience. This is evidenced by the professional artist group reaching a high rating of 4.40. Because they understand how rigorous and meticulous traditional 3D workflows are, they deeply appreciate the support provided by this new tool. Meanwhile, the non-professional group initially scored 2.98 but increased to 4.05 as they became more familiar with the system. They felt excited because they could finally create basic 3D models, a task they previously considered beyond their reach. Despite not fully utilizing advanced features, this group still knew how to leverage joint connections to create motion. One child even compared the experience to the familiarity and fun of

handmade doll making or statue painting, transforming a complex technology into an inspiring creative game.

The system's high responsiveness allows users to see immediate results from their actions, fostering a natural sense of creativity. A significant advantage is the automatic generation of secondary motions, such as gravitational fall and bounce, which enhance realism without requiring the user to manually edit complex individual frames. Furthermore, users expressed significant satisfaction with the flexible control over the stiffness and softness of different character components. Ultimately, these advantages culminated in high overall satisfaction ratings across the board. The professional artists expressed a strong approval rating of 4.26/5.0, while the non-professional group's satisfaction level witnessed a significant increase of approximately 57.31%, surging from an initial 2.53/5.0 due to unfamiliarity to a remarkable 3.98/5.0 once proper instruction was provided.

From a professional perspective, one artist suggested the possibility of linking motions to create more complex sequences, such as bee wings. In this scenario, the wings would automatically follow the body's movement and stop beating when the body is stationary. Another critique noted that the system's motions can sometimes be unpredictable and do not always follow the user's exact intent. While the model is engaging during the ideation phase, this lack of predictability could be a limitation in professional projects that demand absolute precision in timing and positioning. Nevertheless, the artists unanimously agreed that this technology is highly useful, as it preserves the enjoyment of 2D sketching while leveraging 3D environmental advantages, such as varied camera angles and lighting calculations.

The research findings can be summarized by a participating artist's comment. This technology is not intended to replace professional 3D animation software like Blender. Instead, it introduces a new approach to character animation that shares many similarities with the art of puppetry.

4.6 Limitations and Future Work

While the developed framework successfully integrates sketch-based modeling with real-time physics simulation, several geometric and physical limitations remain to be addressed in future research.

- **Lack of Spatial Depth and Volumetric Control:** The primary geometric limitation stems from the system’s reliance on a single 2D projection plane. Unlike the original Teddy system, which supports temporary coordinate rotations for multi-view sketching [2], the current implementation restricts all operations to a flat canvas. Consequently, users cannot add features onto the backside of an existing 3D model, such as drawing a tail on a character’s back, and the inflated volumes are inherently symmetric.
- **Absence of Stroke Layer Ordering:** Compared to systems like MonsterMash [3], the framework does not handle overlapping stroke layers explicitly. Since all generated components are defaulted to the exact same spatial depth, the mesh stitching algorithm may cause topological kinking or abnormal deformation if a user attempts to sketch a limb directly overlapping the torso.
- **Motion Unpredictability and Lack of Manual Weight Control:** From an animation perspective, the passive secondary motions generated by the mass-spring network can occasionally be unpredictable. This limitation in control precision is partly due to the system’s reliance on fully automated harmonic skinning weights, which cannot be modified after calculation. While this automated behavior is engaging for rapid prototyping and playful ideation, it lacks the precision and strict timing required for professional production pipelines that demand absolute keyframe control.

To overcome these constraints, future developments will focus on several key areas. First, a stroke layer ordering mechanism should be implemented to handle complex, overlapping anatomy naturally. Second, integrating an implicit depth estimation model could allow the system to infer asymmetrical volumes from a single viewpoint. Third, developing an interactive weight painting tool is necessary to grant users the ability to manually refine skinning boundaries and customize influence zones. Finally, extending the mass-spring solver to accommodate joint angle limits and user-defined constraints will further bridge the gap between automated physics simulation and predictable artistic intent.

Conclusion

This thesis presents a sketch-based 3D modeling and animation system designed to make the creative process more accessible by reducing complex technical hurdles. By combining geometric methods with physics-based simulation, the research has achieved several results that simplify the transition from a 2D idea to a 3D reality.

The system offers a simplified workflow that allows for the creation of 3D characters from 2D sketches in real time. Rather than requiring the assembly of individual parts, this approach lets users focus on the character's overall silhouette while the system assists with mesh stitching and automatic skeletonization.

Additionally, the integration of a mass-spring system into the hierarchical skeletal structure helps automate the generation of secondary motions. This provides characters with natural physical effects, such as inertia and overlapping action, without requiring the user to perform manual keyframing. Users can also adjust physical properties to simulate various materials, ranging from rigid to soft and squishy.

The study further improves interactivity through features such as direct surface painting, manual skeleton refinement, and path-based animation. Experimental results and user feedback indicate that the system remains responsive and provides immediate feedback, ensuring a smooth experience even as model complexity increases.

While there are still limitations regarding spatial depth and layer ordering compared to professional systems, this work helps bridge the gap between 2D artistic intent and 3D geometric representation. It is hoped that this approach will support the rapid ideation and prototyping of character concepts in a more convenient way.

LIST OF AUTHOR’S PUBLICATIONS

The research results presented in this thesis have been partially published in the following scientific paper:

- [1] Chau Thi Ma, and **Tam Minh Le**. Real-Time Rigging and Secondary Motion for Sketch-Based 3D Characters. *Computational Science and Its Applications – ICCSA 2025 Workshops*, Springer Nature Switzerland, Cham, 213–228, 2025.

Note: The aforementioned paper is the result of collaborative research and development conducted during the graduation thesis process. It serves as the geometric foundation and core structure upon which the author has continuously improved, integrated the physics simulation system, and finalized this thesis.

Bibliography

- [1] Mihaly Csikszentmihalyi. *Flow: The Psychology of Optimal Experience*. 1991.
- [2] Takeo Igarashi, Satoshi Matsuoka, and Hidehiko Tanaka. *Teddy: A Sketching Interface for 3D Freeform Design*. Association for Computing Machinery, New York, NY, USA, 1 edition, 2023.
- [3] Marek Dvoroznak, Daniel Sykora, Brian Curless, Cassidy Jonathan Curtis, Olga Sorkine-Hornung, and David H. Salesin. Monster mash: A single-view approach to casual 3d modeling and animation. pages 1–12, New York, NY, USA, 2020.
- [4] Frank Thomas and Ollie Johnston. *Disney Animation: The Illusion of Life*. Abbeville Press, New York, 1981.
- [5] Chiew-Lan Tai, Hongxin Zhang, and Jacky Fong. Prototype modeling from sketched silhouettes based on convolution surfaces. *Comput. Graph. Forum*, 23:71–84, 03 2004.
- [6] Ryan Schmidt, Brian Wyvill, Mario Costa Sousa, and Joaquim A. Jorge. Shapeshop: Sketch-based solid modeling with blobtrees. In *Eurographics Workshop on Sketch-Based Interfaces and Modeling*, pages 53–62, 2005.
- [7] Andrew Nealen, Takeo Igarashi, Olga Sorkine, and Marc Alexa. Fibermesh: Designing freeform surfaces with 3d curves. *ACM Transactions on Graphics*, 26(3):41, 2007.
- [8] Adrien Bernhardt, Adeline Pihuit, Marie-Paule Cani, and Loïc Barthe. Matisse: Painting 2d regions for modeling free-form shapes. In *Eurographics Workshop on Sketch-Based Interfaces and Modeling*, pages 57–64, 2008.
- [9] Yotam Gingold, Takeo Igarashi, and Denis Zorin. Structured annotations for 2d-to-3d modeling. *ACM Transactions on Graphics*, 28(5):148, 2009.
- [10] Alec Rivers, Takeo Igarashi, and Frédo Durand. 2.5d cartoon models. *ACM Transactions on Graphics*, 29(4):59, 2010.
- [11] Olga A. Karpenko and John F. Hughes. Smoothsketch: 3d free-form shapes from complex sketches. *ACM Transactions on Graphics*, 25(3):589–598, 2006.

- [12] Zhaoliang Lun, Matheus Gadelha, Evangelos Kalogerakis, Subhansu Maji, and Rui Wang. 3d shape reconstruction from sketches via multi-view convolutional networks. In *Proceedings of International Conference on 3D Vision*, pages 67–77, 2017.
- [13] Johanna Delanoy, Mathieu Aubry, Phillip Isola, Alexei A. Efros, and Adrien Bousseau. 3d sketching using multi-view deep volumetric prediction. *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, 1(1):21, 2018.
- [14] Changjian Li, Hao Pan, Yang Liu, Alla Sheffer, and Wenping Wang. Robust flow-guided neural prediction for sketch-based freeform surface modeling. *ACM Transactions on Graphics*, 37(6):238, 2018.
- [15] Petru Borosán, Thomas Auzinger, and Michael Wimmer. Rigmesh: Automatic rigging for articulated characters. *Computer Graphics Forum*, 31(2pt3):381–390, 2012.
- [16] Ilya Baran and Jovan Popović. Pinocchio: Automatic skeleton generation for 3d models. *ACM Transactions on Graphics (TOG)*, 26(3):33, 2007.
- [17] Zhan Xu, Yang Zhou, Evangelos Kalogerakis, Chris Landreth, and Karan Singh. Rignet: Neural rigging for articulated characters. In *ACM Transactions on Graphics (TOG)*, volume 39, page 67. ACM, 2020.
- [18] Ji Yang, Xinxin Zuo, Sen Wang, Zhenbo Yu, Xingyu Li, Bingbing Ni, Minglun Gong, and Li Cheng. Object wake-up: 3d object rigging from a single image. *arXiv preprint arXiv:2108.02708*, 2021.
- [19] Adam W. Bargteil and Tamar Shinar. An introduction to physics-based animation. In *ACM SIGGRAPH 2019 Courses*, SIGGRAPH ’19, New York, NY, USA, 2019. Association for Computing Machinery.
- [20] Xavier Provot. Deformation constraints in a mass-spring model to describe rigid cloth behaviour. In *Proceedings of Graphics Interface ’95, GI ’95*, pages 147–154, Toronto, Ontario, Canada, 1995. Canadian Human-Computer Communications Society.
- [21] Robert Bridson, Ronald Fedkiw, and John Anderson. Robust treatment of collisions, contact and friction for cloth animation. *ACM Trans. Graph.*, 21(3):594–603, July 2002.

- [22] Matthias Teschner, Bruno Heidelberger, Matthias Müller, and Markus Gross. A versatile and robust model for geometrically complex deformable solids. volume 0, pages 312–319, 07 2004.
- [23] Andrew Selle, Michael Lentine, and Ronald Fedkiw. A mass spring model for hair simulation. *ACM Trans. Graph.*, 27(3):1–11, aug 2008.
- [24] Robert E Rosenblum, Wayne E Carlson, and Edwin Tripp III. Simulating the structure and dynamics of human hair: modelling, rendering and animation. *The Journal of Visualization and Computer Animation*, 2(4):141–148, 1991.
- [25] Tiantian Liu, Adam W. Bargteil, James F. O’Brien, and Ladislav Kavan. Fast simulation of mass-spring systems. *ACM Trans. Graph.*, 32(6), November 2013.
- [26] Guanxiong Chen, Shashwat Suri, Yuhao Wu, Yixian Cheng, Ganidhu Abeysirigoonawardena, Etienne Vouga, David I. W. Levin, and Dinesh K. Pai. Springtime: Learning simulatable models of cloth with spatially-varying constitutive properties, 2026.
- [27] Artur Grigorev, Bernhard Thomaszewski, Michael J. Black, and Otmar Hilliges. Hood: Hierarchical graphs for generalized modelling of clothing dynamics, 2023.
- [28] Yang Zheng, Qingqing Zhao, Guandao Yang, Wang Yifan, Donglai Xiang, Florian Dubost, Dmitry Lagun, Thabo Beeler, Federico Tombari, Leonidas Guibas, and Gordon Wetzstein. Physavatar: Learning the physics of dressed 3d avatars from visual observations, 2024.
- [29] D. Halliday, R. Resnick, and J. Walker. *Fundamentals of Physics*. Halliday & Resnick Fundamentals of Physics. John Wiley & Sons Canada, Limited, 2010.
- [30] Sebastian Martin, Bernhard Thomaszewski, Eitan Grinspun, and Markus Gross. Example-based elastic materials. *ACM Trans. Graph.*, 30(4), July 2011.
- [31] David Baraff and Andrew Witkin. *Large Steps in Cloth Simulation*. Association for Computing Machinery, New York, NY, USA, 1 edition, 2023.
- [32] Pushkar Joshi, Mark Meyer, Tony DeRose, Brian Green, and Tom Sanocki. Harmonic coordinates for character articulation. *ACM Trans. Graph.*, 26(3):71–es, July 2007.

- [33] Miles Macklin, Kier Storey, Michelle Lu, Pierre Terdiman, Nuttapong Chentanez, Stefan Jeschke, and Matthias Müller. Small steps in physics simulation. In *Proceedings of the 18th Annual ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, SCA '19, New York, NY, USA, 2019. Association for Computing Machinery.

Appendix A

User Experience Survey Using Likert Scale

Instructions: Please mark (x) in the column that best represents your experience on a scale of 1 to 5.

(1: *Strongly Disagree* | 2: *Disagree* | 3: *Neutral* | 4: *Agree* | 5: *Strongly Agree*)

No.	Survey Question	1	2	3	4	5
I. Ease of Use						
1	The system interface is intuitive and easy to understand from the first use.					
2	The drawing and modeling tools function exactly as expected.					
3	The system's response speed during model construction is very fast.					
4	Correcting or deleting mistaken strokes is performed easily.					
5	The workflow from sketching to character animation is not overly complex.					
6	The system extracts object boundaries automatically and accurately.					
7	The system provides immediate feedback after every operation.					
8	Layouts are designed to be easily recognizable and memorable.					
9	Connecting body parts (arms, legs, torso) feels natural.					
10	I encountered no difficulty when working with open or closed strokes.					

No.	Survey Question	1	2	3	4	5
II. Creative Experience and Interaction						
11	Drawing and coloring directly onto 3D volumes is an engaging experience.					
12	The automatic rigging helps save significant time in animation setup.					
13	The click-and-drag operations to create movement are smooth.					
14	Character movements are vivid thanks to the integrated physical effects.					
15	Adjusting the stiffness makes the character's movement feel more realistic.					
16	The system preserves the spirit and style of the original sketch.					
17	I feel comfortable experimenting with new and creative drawing ideas.					
18	Joint areas deform naturally without breaking or overlapping.					
19	I can flexibly control the physical properties of each individual part.					
20	The system creates a natural interaction, similar to puppeteering.					

No.	Survey Question	1	2	3	4	5
III. Level of Satisfaction						
21	This system is truly useful for rapid ideation and sketching.					
22	The quality of the generated 3D models meets my requirements.					
23	I would like to use this tool to create characters for games/animations.					
24	This tool makes approaching 3D graphics feel more accessible.					
25	I will continue to use this system in future projects.					
26	The system's computational performance is stable on my device.					
27	Surface drawing features work effectively.					
28	I am satisfied with the level of automation provided by the system.					
29	I will recommend this system to other interested users.					
30	Overall, the creative experience on the system is very positive.					